Exam 1

- Covers six assigned papers on the syllabus: Entity-Relationship Model, Gamma, FoundationDB, Hybrid Range Partitioning Strategy, BG Benchmark, Grasper.
- Closed book, closed notes, no cheat notes, cannot call a friend!
- Bring pencil/pen and eraser. We provide exam paper and extra pages as necessary.
- October 12:
  - 5-6 pm: Review for Exam 1
  - 6-7 pm: Exam 1 (those of you with exemption may have 30 more minutes)

Grasper: A Distributed System for Property Graphs Chen et. al.

Shahram Ghandeharizadeh
CSCI 585

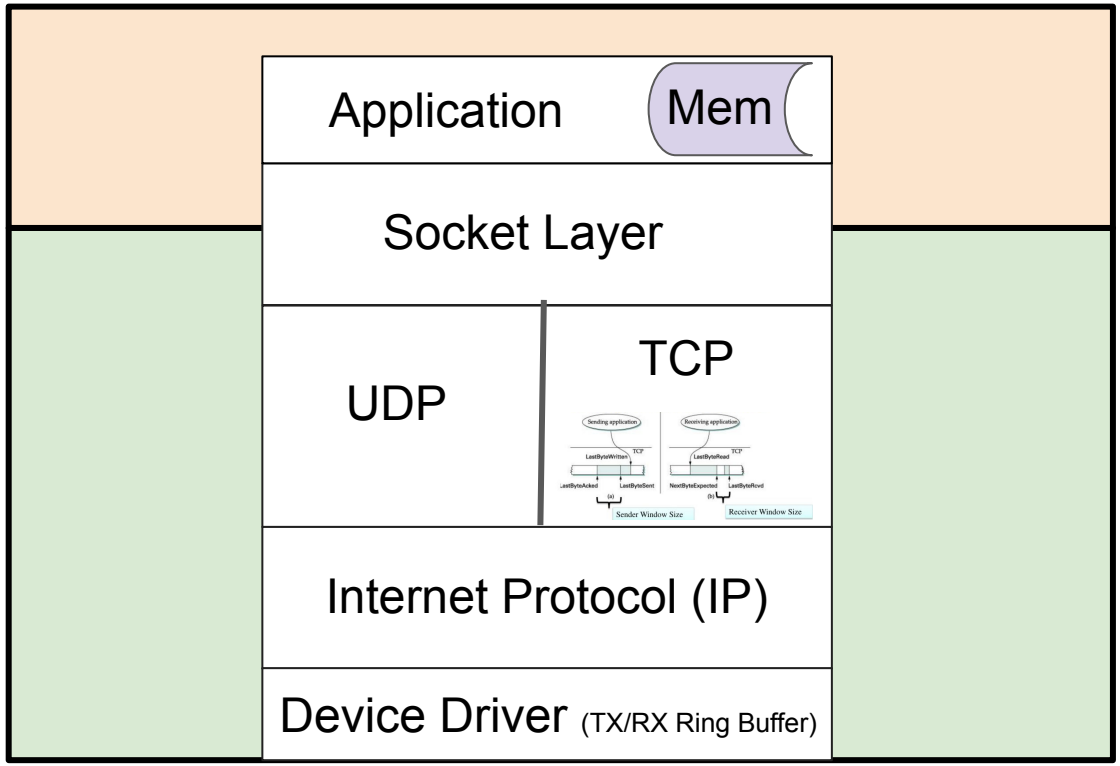
Outline

- Background
- Property Graph (PG) data model: Vertices, edges, properties
- PG query language: Gremlin
- Grasper query processor
- Conclusions

Ethernet

User Memory

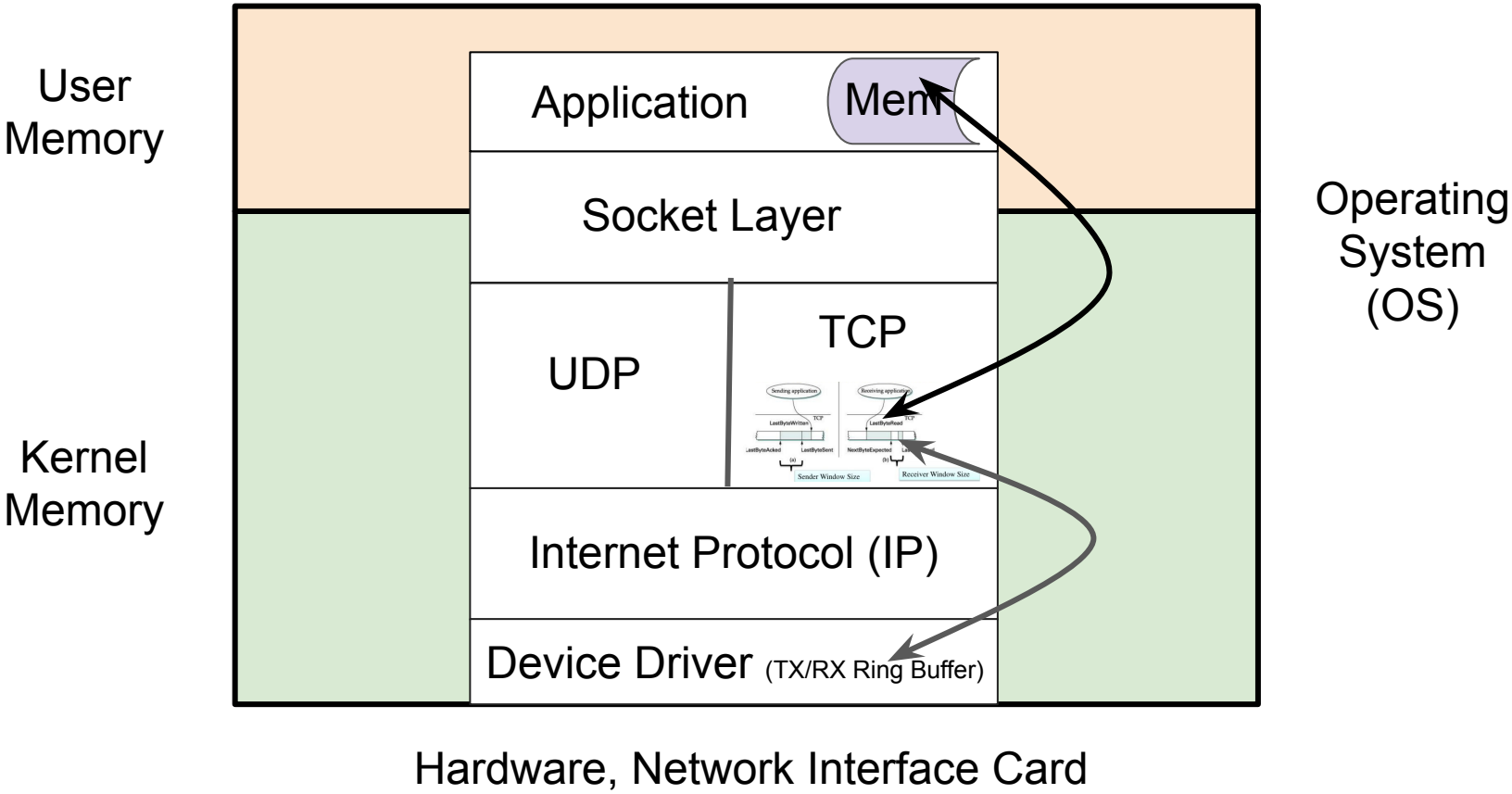
Kernel Memory



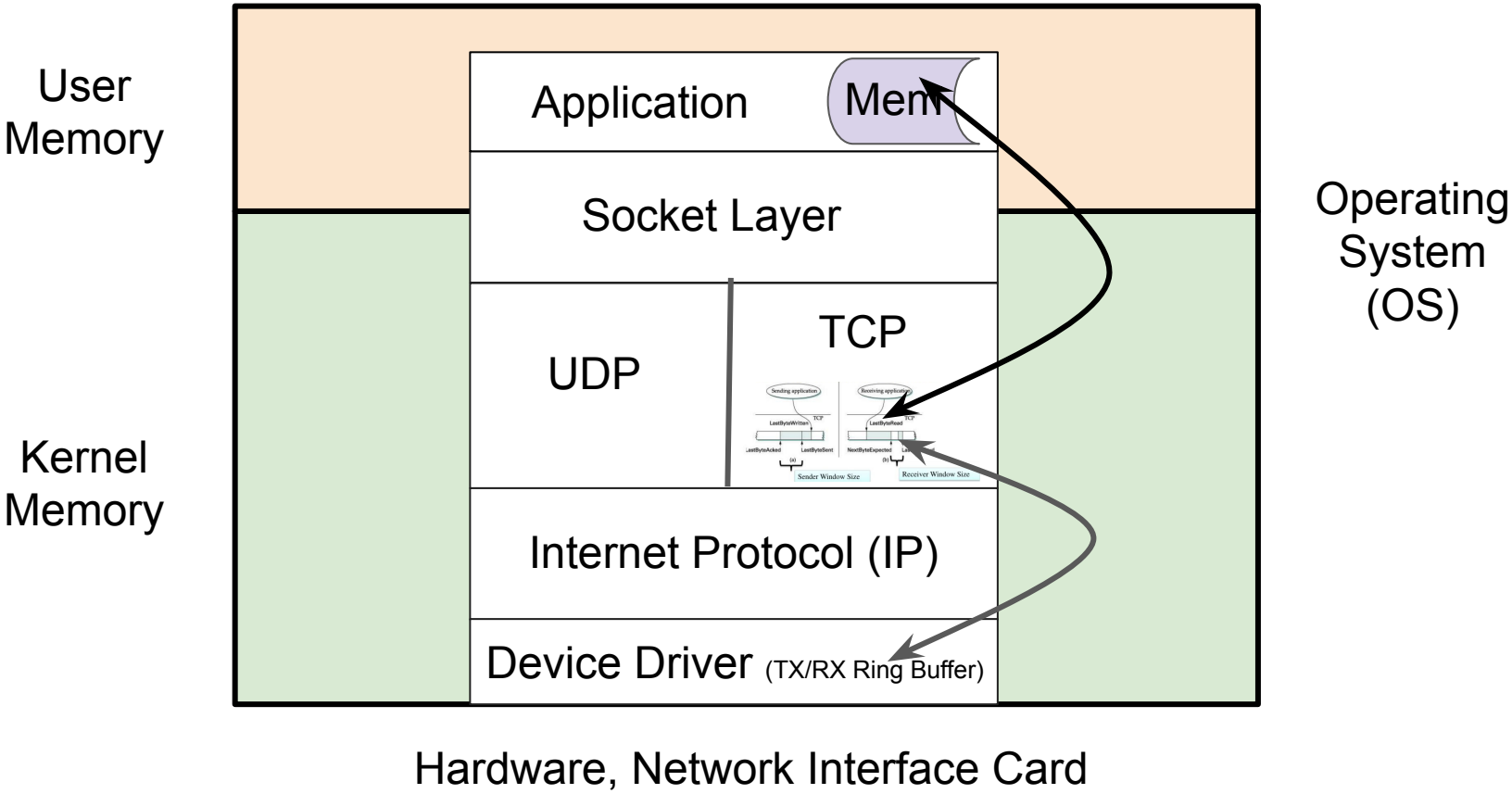
Operating System (OS)

Hardware, Network Interface Card

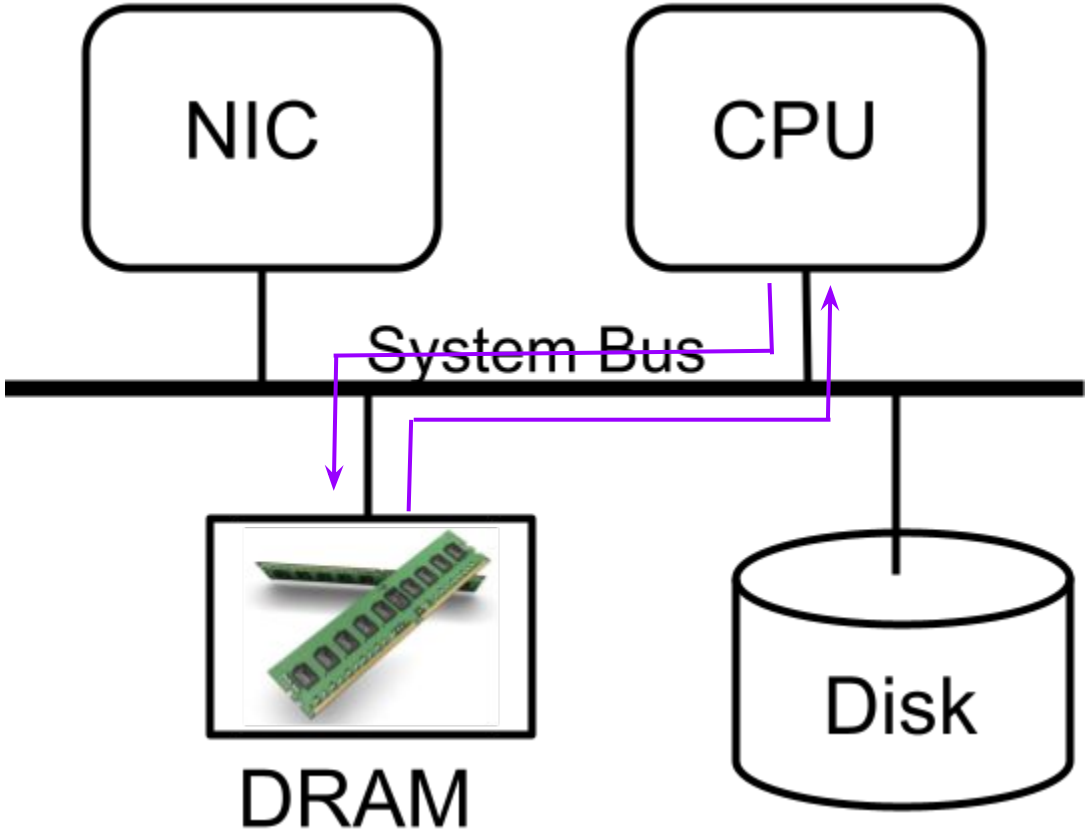
Ethernet: Copies Data 4x, 2x at Sender + 2x at Receiver



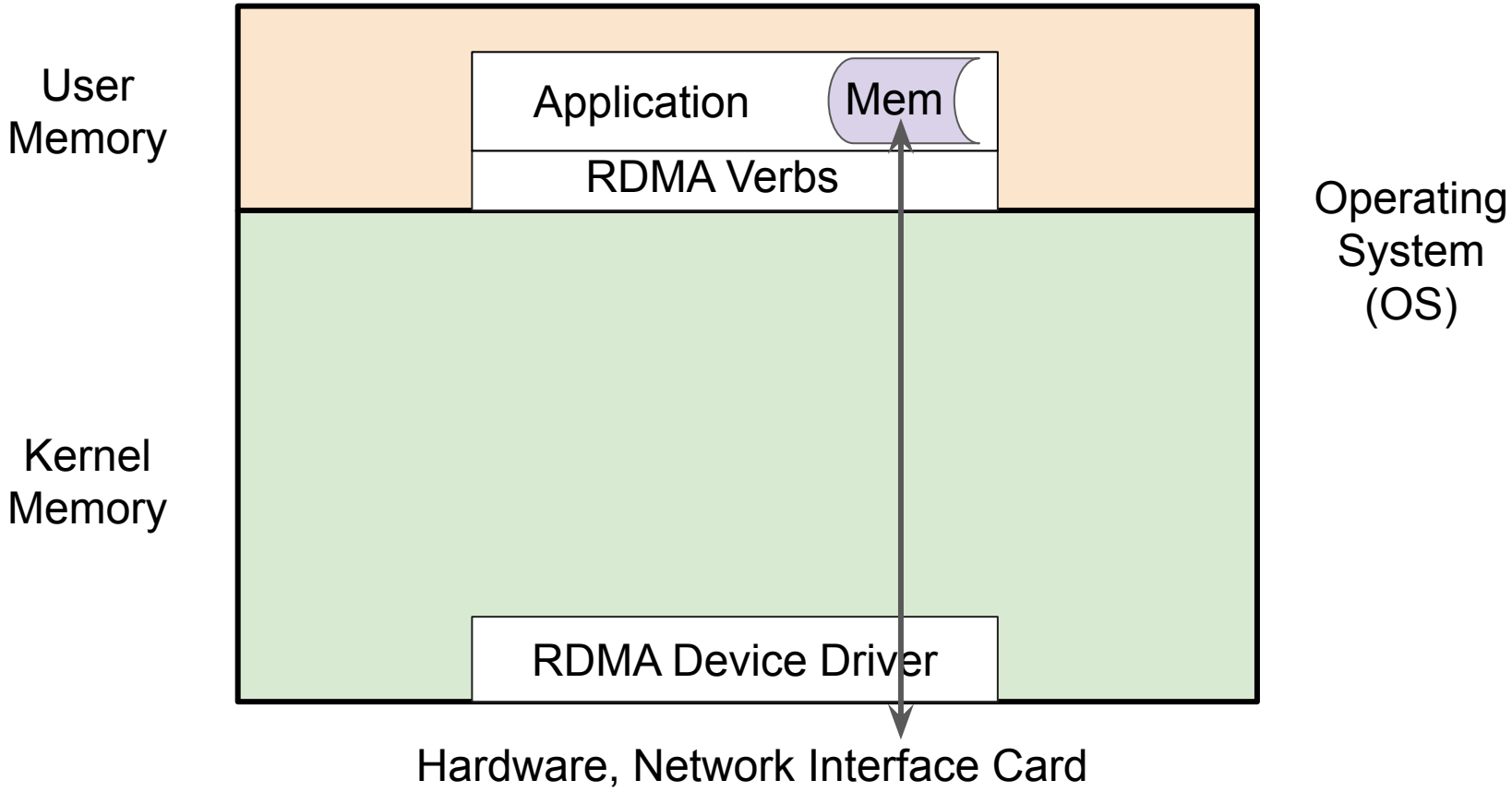
Ethernet: Copies Data 4x, 2x at Sender + 2x at Receiver



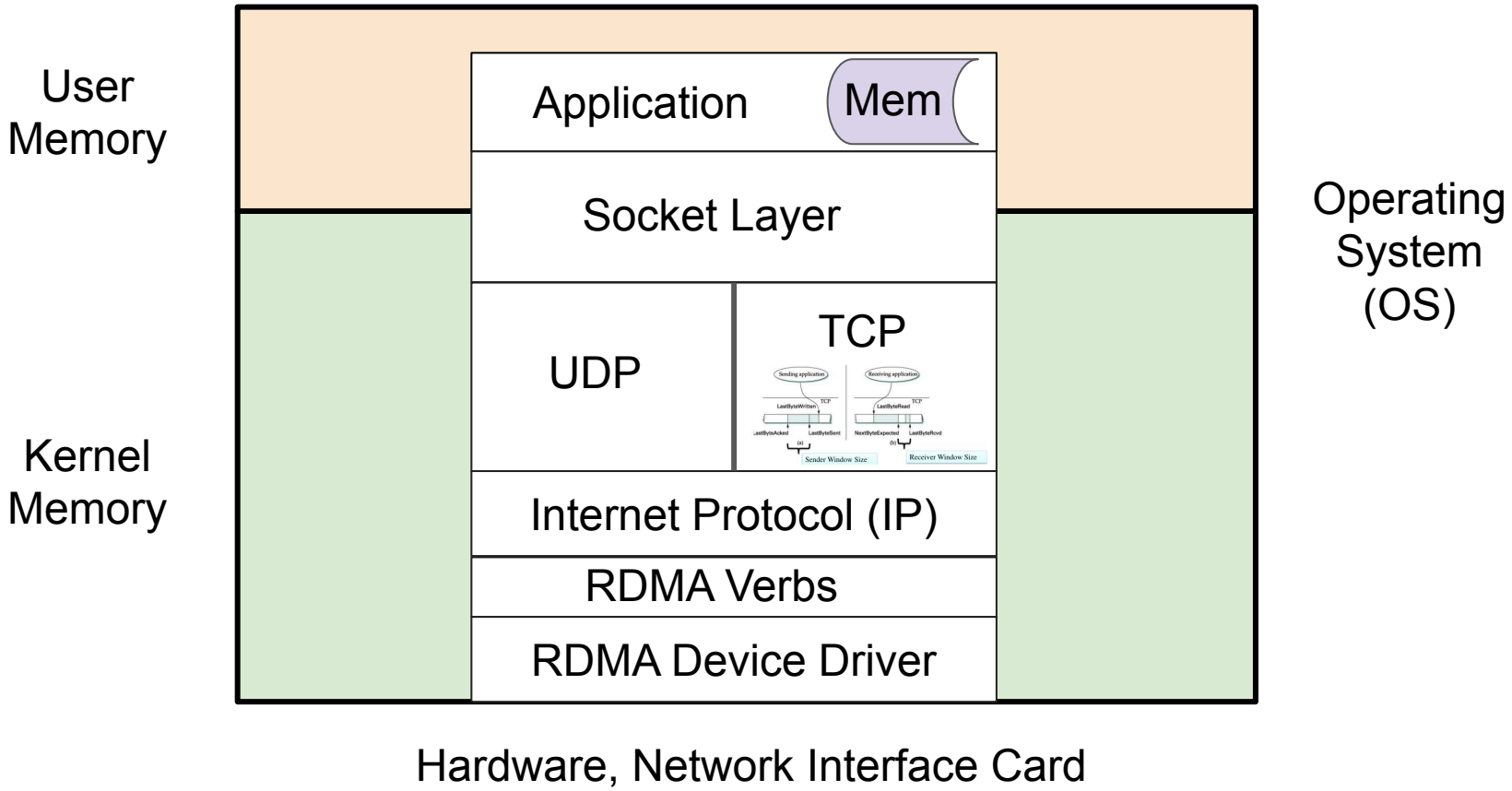
Copy Block 1 to Block 2



RDMA: Zero Copy



IPoIB: Copies Data 4x, 2x at Sender, 2x at Receiver



RDMA

- Remote Direct Memory Access (RDMA) bypasses the OS and supports zero-copy transmission. It minimizes latency and maximizes throughput. It can be used in two ways:
  - RDMA commands classified as two-sided send/recv verbs and one-sided read/write verbs. One-sided verbs enable a sender node to write its message to the memory address of a destination node.
  - IPoIB, IP over Infiniband

IPoIB (IP over InfiniBand), RDMA can achieve at least an order of magnitude lower round-trip latencies (< 10us) when the payload is small (i.e., < 4K bytes). In addition, the latency of RDMA is relatively insensitive to the increase in the payload size for small-sized messages, as long as the network is not saturated. To be more specific, in our RDMA-enabled cluster, the latency of one-sided read operation increases only 2.2X (from 3.94us to 8.63us) when the payload size increases 512X (from 8 bytes to 4K bytes). When the payload continues to grow (> 4K bytes), the latency of RDMA starts to increase linearly, which is similar to Ethernet and IPoIB, but RDMA is still an order of magnitude faster than Ethernet and >2X faster than IPoIB. RDMA also achieves much higher throughput under all payload sizes, which is about 10X higher than Ethernet and IPoIB for both small (< 4K bytes) and large payloads (> 4K bytes).

RDMA

- Remote Direct Memory Access (RDMA) bypasses the OS and supports zero-copy transmission. It minimizes latency and maximizes throughput. It can be used in two ways:
  - RDMA commands classified as two-sided send/recv verbs and one-sided read/write verbs. One-sided verbs enable a sender node to write its message to the memory address of a destination node.
  - IPoIB, IP over Infiniband

What does "order of magnitude" mean?

IPoIB (IP over InfiniBand), RDMA can achieve at least an order of magnitude lower round-trip latencies (< 10us) when the payload is small (i.e., < 4K bytes). In addition, the latency of RDMA is relatively insensitive to the increase in the payload size for small-sized messages, as long as the network is not saturated. To be more specific, in our RDMA-enabled cluster, the latency of one-sided read operation increases only 2.2X (from 3.94us to 8.63us) when the payload size increases 512X (from 8 bytes to 4K bytes). When the payload continues to grow (> 4K bytes), the latency of RDMA starts to increase linearly, which is similar to Ethernet and IPoIB, but RDMA is still an order of magnitude faster than Ethernet and >2X faster than IPoIB. RDMA also achieves much higher throughput under all payload sizes, which is about 10X higher than Ethernet and IPoIB for both small (< 4K bytes) and large payloads (> 4K bytes).

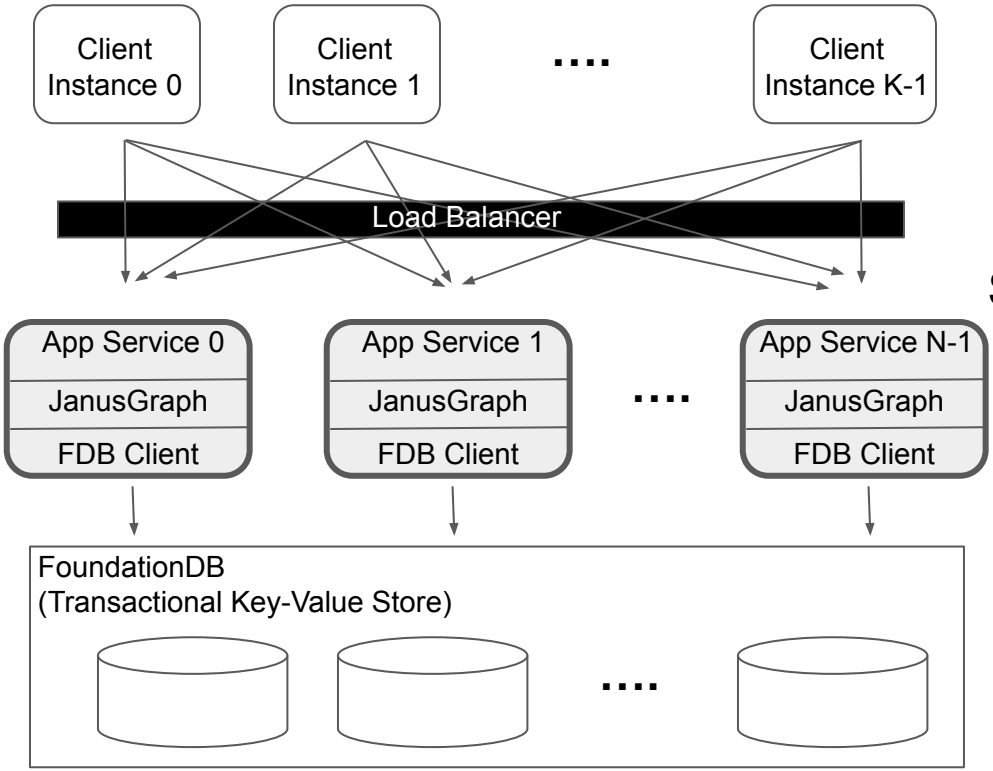
Ethernet & RDMA: NIC Price (Oct 5, 2026)





	Ethernet	RDMA
1 Gbps	$12-$17	
2.5-5 Gbps	$17-$20	
10 Gbps	$80-$110	$90-$120
25-50 Gbps		$120-$250
100 Gbps		$340-$750
200+ Gbps		$1200-$1500+

Separation of Data Store from Data Model: JanusGraph



Stateless Graph Data Model

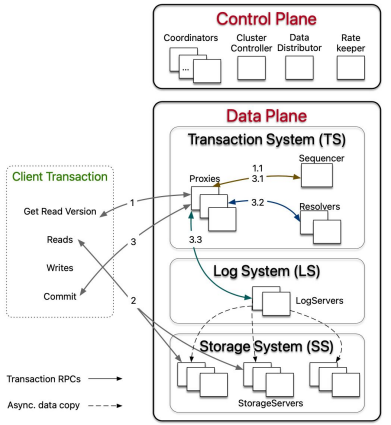


Figure 1: The architecture and the transaction processing of FDB.

Grasper: Stateful Processing

Stateful Graph Data Model

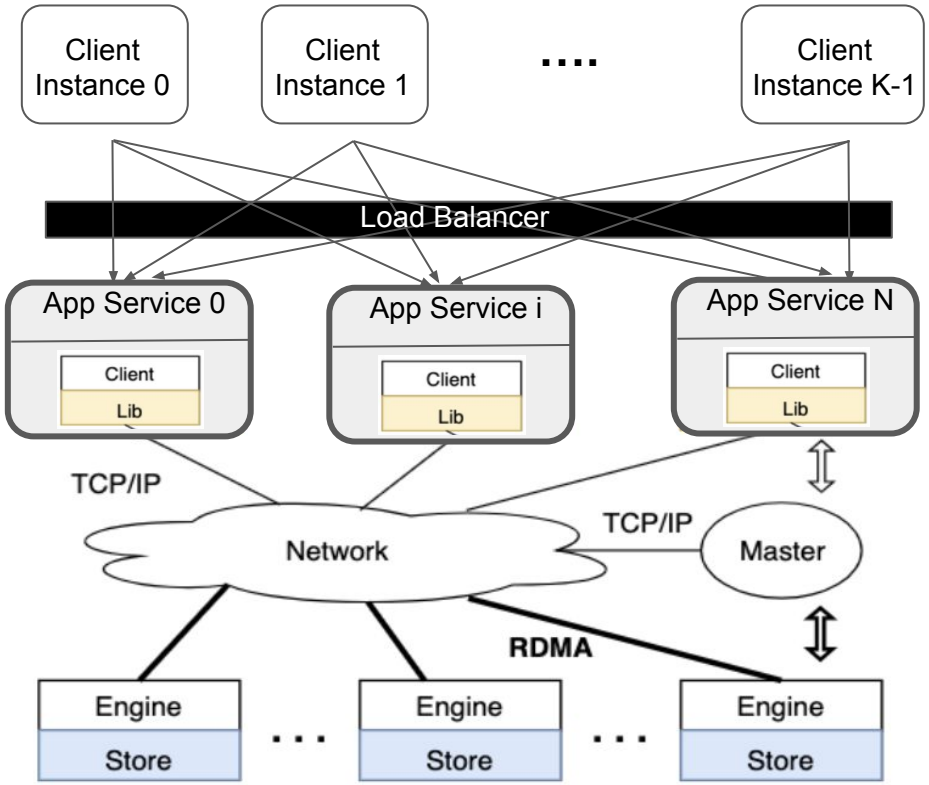


Figure 4: The architecture of Grasper

Grasper: Stateful Processing

RDMA and TCP/IP on one network?

Stateful Graph Data Model

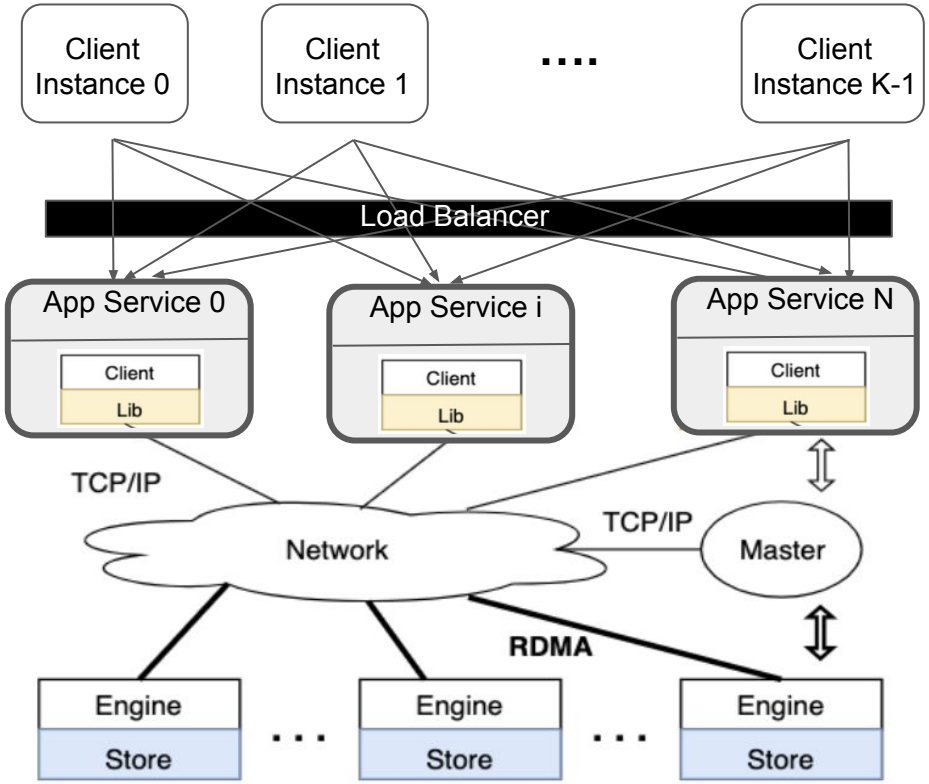


Figure 4: The architecture of Grasper

Why separate storage from processing?

Scale one independent of the other.

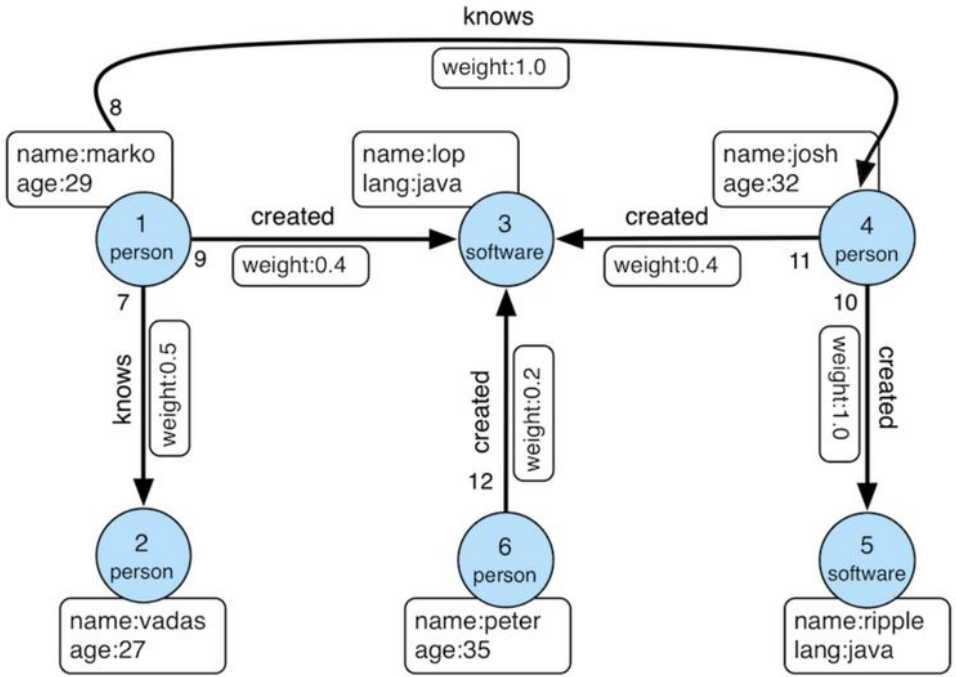
Outline

- Background
- Property Graph (PG) data model: Vertices, edges, properties
- PG query language: Gremlin
- Grasper query processor
- Conclusions

Property Graph (PG) Data Model

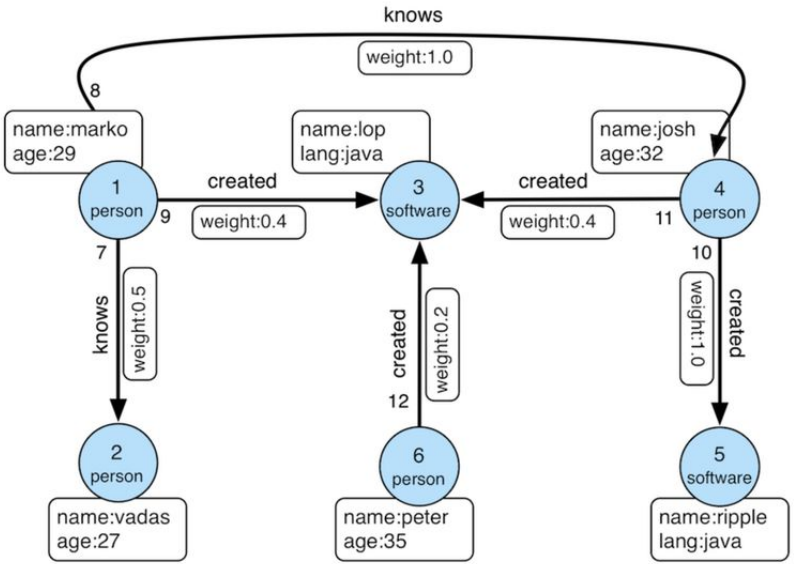
A property graph (PG) consists of a set of vertices V and a set of edges E, where vertices are entities and an edge models the relationship between two entities.

Modern Graph



Vertices and Edges

Each vertex/edge has a label to represent its role and a set of properties to describe its features in the form of key-value pairs. No fixed schema: different vertices/edges may have different properties. Edges are directed. Vertices typically specify properties while edges have quantitative properties.



Numerous Systems

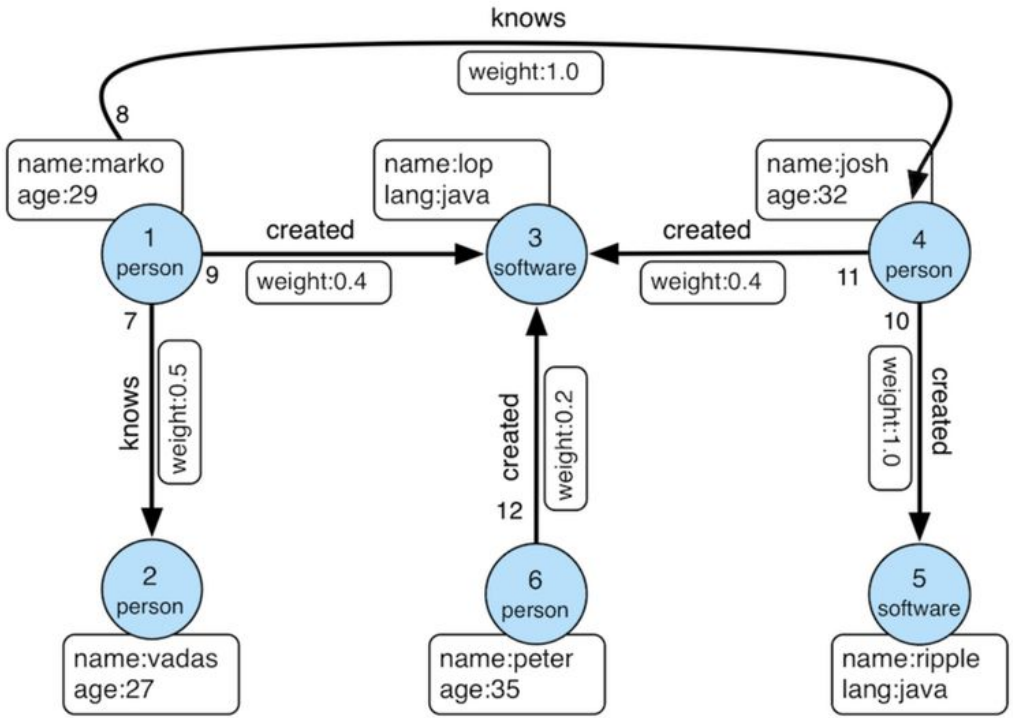
- Commercial systems:
  - Neo4j: Cypher Query Language, CQL
  - Grasper, JanusGraph, TinkerPop: Gremlin
  - Jena, Wukong (RDMA), Virtuoso for RDF Data: SPARQL
    - RDF models the knowledge graph as a list of triplets (Subject, Predicate, Object)
    - Logically RDF and PG can be converted into each other.
- Gremlin is a procedural query language: Programmer must specify both "what" data is desired and "how" the system should retrieve it.
  - It consists of a set of query steps or pipes.
  - Gremlin is trivial to use.
- Gremlin may be used to author both simple and complex queries.
  - A simple query processes a few vertices/edges using index structures.
  - A complex query processes a large number of vertices/edge - potentially the whole graph.

Gremlin Query Language

- To learn Gremlin: Visit and read PracticalGremlin
- Get started and have fun with it:
  - Download the Gremlin Console from TinkerPop
  - Download the air-routes graph database from klawrence/graph github repository: air-routes-latest.graphml
  - Launch the Gremlin console and load the airport database:
    - graph = TinkerGraph.open()
    - g=graph.traversal()
    - graph.io(IoCore.graphml()).readGraph('./data/air-routes-latest.graphml')
  - Verify the database exists:
    - g.V().count()
    - g.E().count()
  - To load the "Modern Graph":
    - graph = TinkerFactory.createModern()
    - g = graph.traversal()
    - g.V().valueMap()
    - g.E().valueMap()

Let's Have Some FUN!

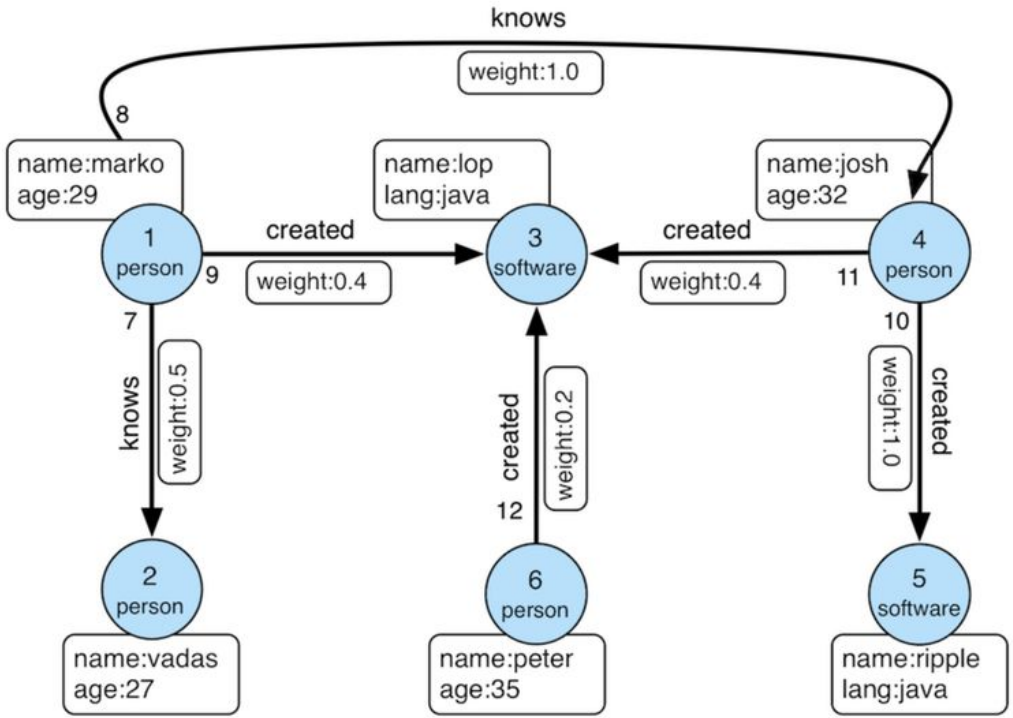
- Who does Marko know?
  - Find Marko's vertex.
  - Find all the "knows" edges going out of Marko's vertex.
  - Get the property of the resulting vertices.




```
g.V().has('name','marko').out('knows').valueMap()
```

Who knows Marko?

1. Find the vertex corresponding to Marko.
2. Find all vertices with edges incoming to Marko's vertex.
3. Fetch the properties of the vertices in Step 2.

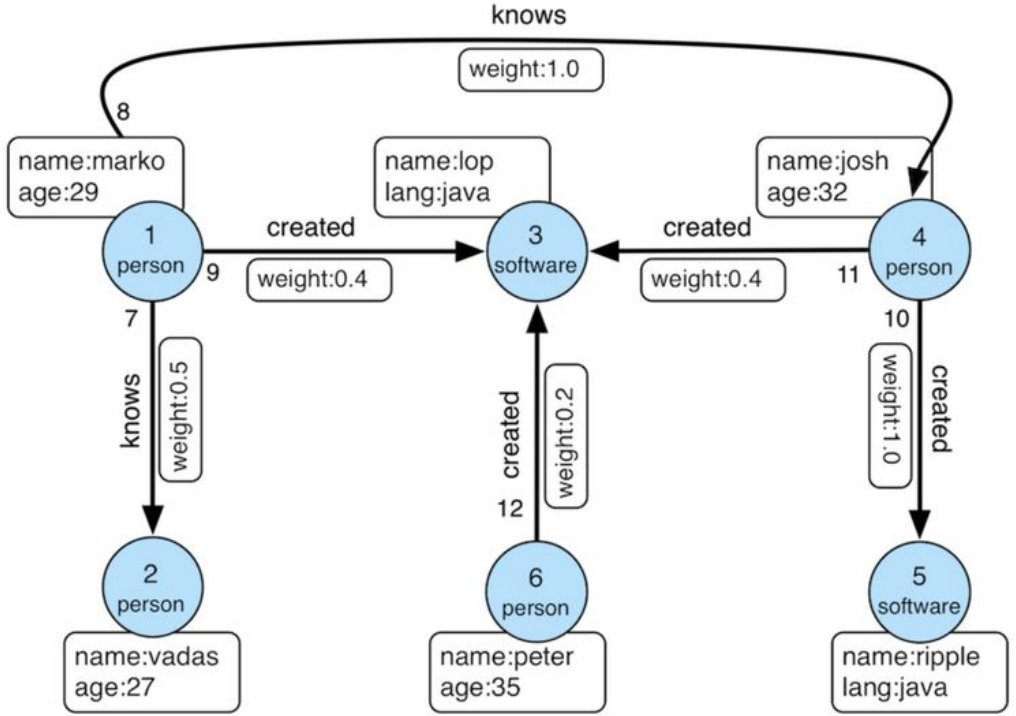



```
g.V().has('name','marko').in('knows').valueMap()
```

Who has created a Java Program?

Find all vertices with the language 'java'.

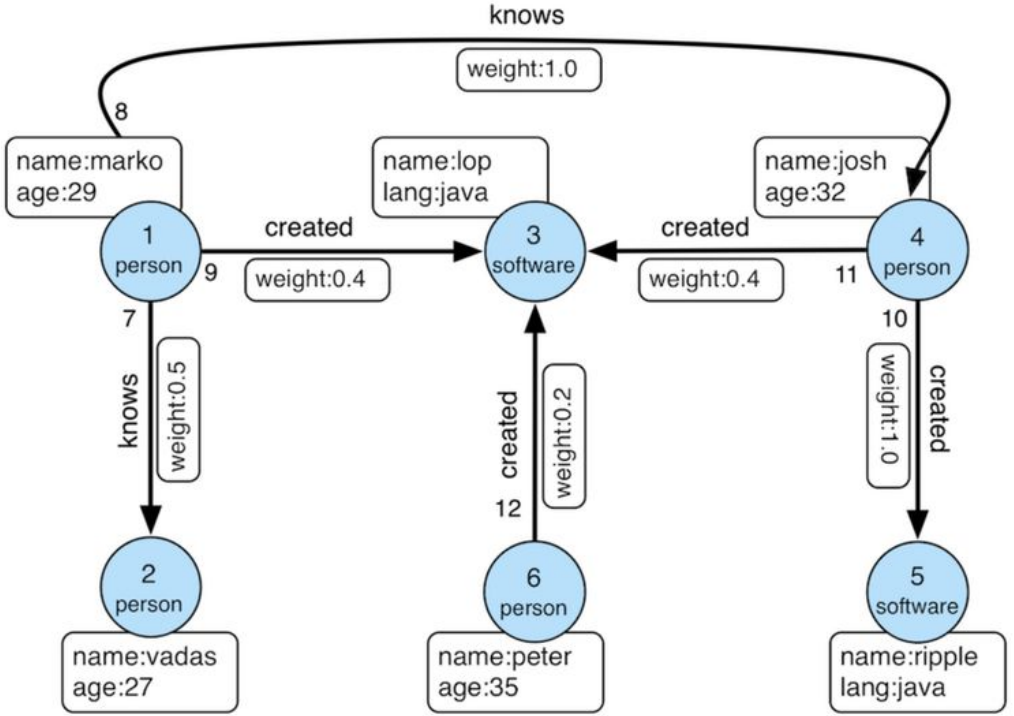
For each vertex, compute those vertices with incoming edges labeled as 'created'.




```
g.V().has('lang','java').in('created').valueMap().dedup()
```

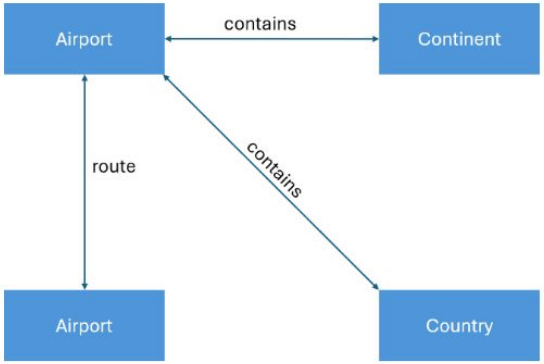
Top person who created most software?

For each vertex, count the number of outgoing edges labeled 'created' in descending order.




```
g.V().order().by( __.outE().hasLabel('created').count(), desc) .valueMap()
```

Top 3 airports with most outgoing flights?




```
g.V().hasLabel('airport').
    order().by( __.out('route').count(), desc ).
    project('airportName', 'outgoingFlights').by('code').
    by(__.out('route').count()).limit(3)
```

Complex eBay Queries

```
g.V().has("User","id",?).
    local(__.outE("useSigninIpAttempt").order().by("
        lastModifiedTime",Order.desc).limit(64L).has
        ("lastModifiedTime",?)).inV().dedup().
    local(__.inE("useSigninIpAttempt").order().by("
        lastModifiedTime",Order.desc).limit(64L).has
        ("lastModifiedTime",?)).outV().elementMap("
        lastSuspensionTime").dedup().
    where(__.select("lastSuspensionTime").is(?)).
        count()
```

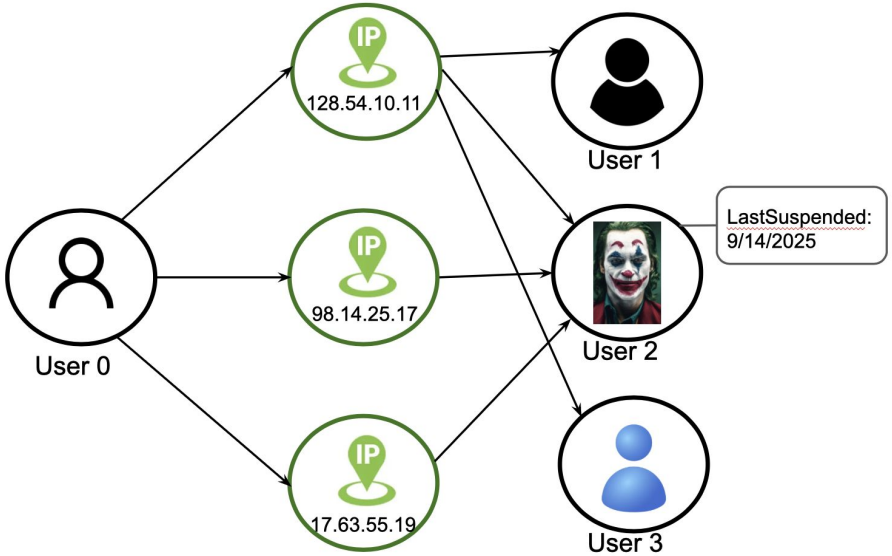
```
g.union(
    __.V().has(\"graph_id\",?).has(\"entity_id\",?).
        limit(1L)).coalesce(__.local(__.outE(?).has
        (\"probability\",P.gt(?)).order().by(\"
        probability\",Order.desc)).project(\"
        destinationProps\",\"edgeProps\",\"
        sourceProps\").by(__.inV().valueMap(\"
        entity_id\",\"count\",\"prefLabel\",\"type
        \",\"image_uri\")).by(__.valueMap(\"type
        \",\"probability\")).by(__.outV().valueMap
        (\"entity_id\",\"count\",\"prefLabel\",\"
        type\",\"image_uri\")),__.project(\"
        sourceProps\").by(__.valueMap(\"entity_id
        \",\"count\",\"prefLabel\",\"type\",\"
        image_uri\"))
)
```

```
g.V().match(
    __.as("VendorAPI").has("vtype","ebayontl:
        VendorAPI"),
    __.as("VendorAPI").out("ebayontl:
        belongToInfohubDomainRelatedToVendorAPI").as
        ("InfohubDomain"),
    __.as("VendorAPI").out("ebayontl:
        hasContractInformationRelatedToVendorAPI").
        as("ContractInformation"),
    __.as("ContractInformation").values("ebayontl:
        ContractInformationId").as("
        ContractInformationId"),
    __.as("InfohubDomain").values("ebayontl:
        InfohubDomainId").as("InfohubDomainId")
).select("ContractInformationId","InfohubDomainId")
```

Fraud Detection

- Given a user, say User 0, check whether the user is suspicious via IP address linking.

```
g.V().has("User","id",?).
    local(__.outE("useSigninIpAttempt").order().by("
        lastModifiedTime",Order.desc).limit(64L).has
        ("lastModifiedTime",?)).inV().dedup().
    local(__.inE("useSigninIpAttempt").order().by("
        lastModifiedTime",Order.desc).limit(64L).has
        ("lastModifiedTime",?)).outV().elementMap("
        lastSuspensionTime").dedup().
    where(__.select("lastSuspensionTime").is(?)).
        count()
```

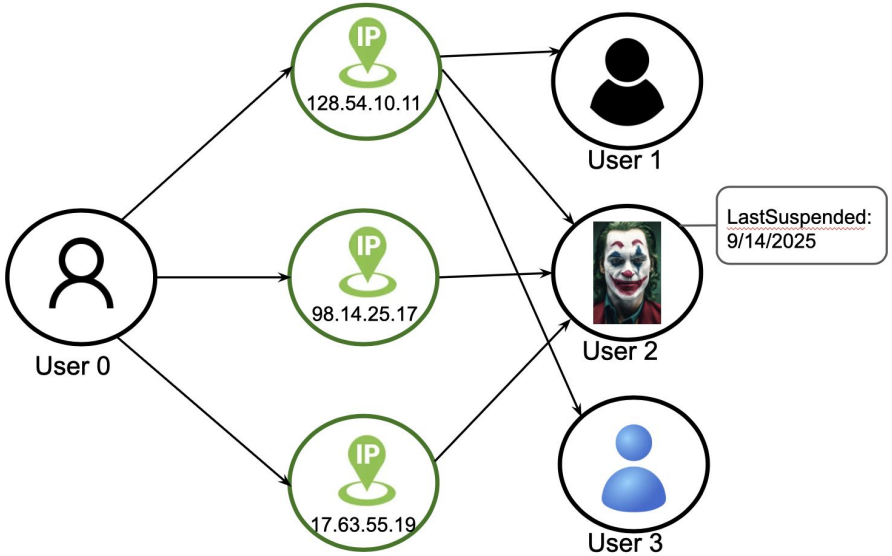


95th percentile latency should be faster than 50 milliseconds.

Fraud Detection: 95th Latency Faster than 50 milliseconds

- Given a user, say User 0, check whether the user is suspicious via IP address linking.

```
g.V().has("User","id",?).
    local(__.outE("useSigninIpAttempt").order().by("
        lastModifiedTime",Order.desc).limit(64L).has
        ("lastModifiedTime",?)).inV().dedup().
    local(__.inE("useSigninIpAttempt").order().by("
        lastModifiedTime",Order.desc).limit(64L).has
        ("lastModifiedTime",?)).outV().elementMap("
        lastSuspensionTime").dedup().
    where(__.select("lastSuspensionTime").is(?)).
        count()
```

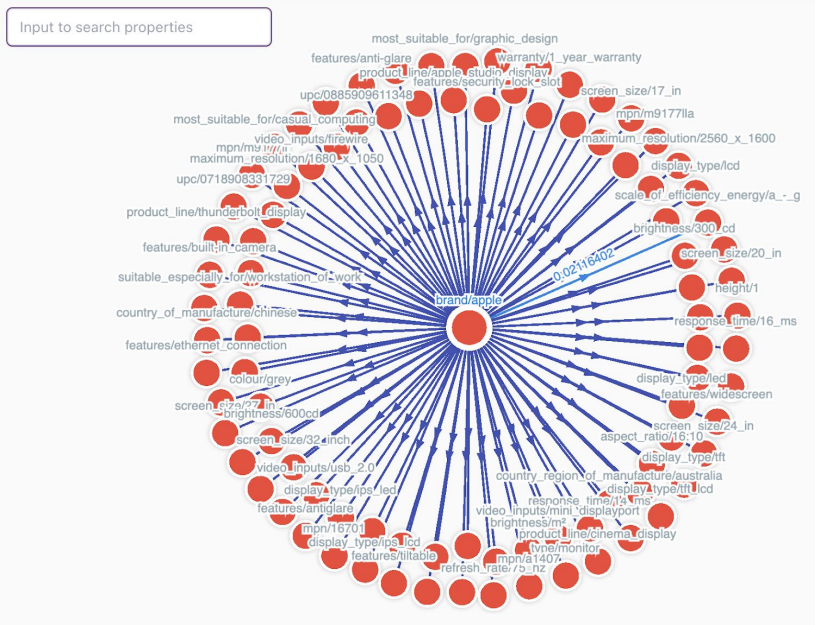


The same query may use other properties: Address, Email, Device, Payment, Phone, Goverment issued ID, etc.

Search/Recommendation

When a user views a specific item, such as an iPhone, the query generates a list of alternative models (such as the Google Pixel or Samsung Galaxy) as personalized recommendations.

```
g.union(
    __.V().has(\"graph_id\",?).has(\"entity_id\",?).
        limit(1L)).coalesce(__.local(__.outE(?).has
        (\"probability\",P.gt(?)).order().by(\"
        probability\",Order.desc)).project(\"
        destinationProps\",\"edgeProps\",\"
        sourceProps\").by(__.inV().valueMap(\"
        entity_id\",\"count\",\"prefLabel\",\"type
        \",\"image_uri\")).by(__.valueMap(\"type
        \",\"probability\")).by(__.outV().valueMap
        (\"entity_id\",\"count\",\"prefLabel\",\"
        type\",\"image_uri\")),__.project(\"
        sourceProps\").by(__.valueMap(\"entity_id
        \",\"count\",\"prefLabel\",\"type\",\"
        image_uri\"))
)
```

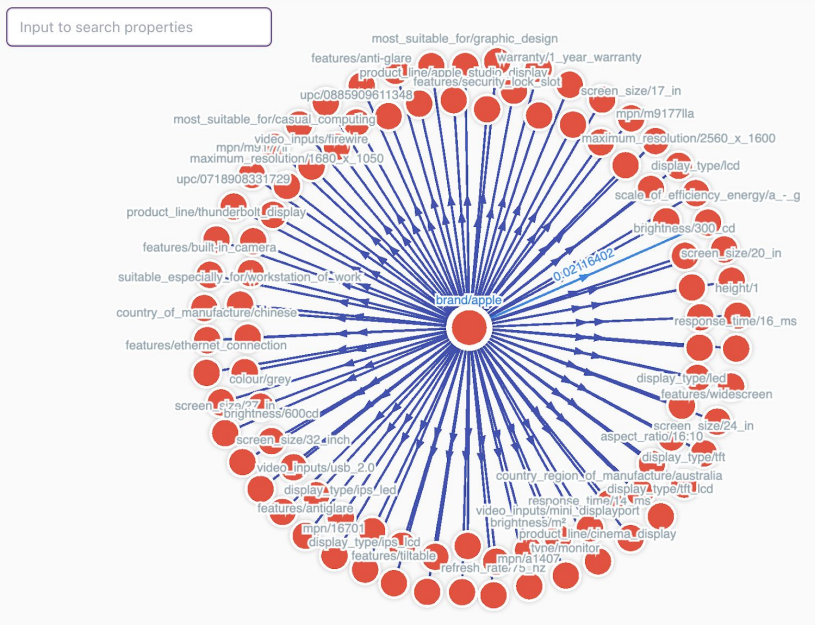


- Middle vertex: Found via (graph_id, entity_id), e.g., iPhone.
- Other vertices are connected to the middle vertex via edges; "weight" on edges expressed in the "probability" property.

Search/Recommendation: 99th latency faster than 50 msec

When a user views a specific item, such as an iPhone, the query generates a list of alternative models (such as the Google Pixel or Samsung Galaxy) as personalized recommendations.

```
g.union(
    __.V().has(\"graph_id\",?).has(\"entity_id\",?).
        limit(1L)).coalesce(__.local(__.outE(?).has
        (\"probability\",P.gt(?)).order().by(\"
        probability\",Order.desc)).project(\"
        destinationProps\",\"edgeProps\",\"
        sourceProps\").by(__.inV().valueMap(\"
        entity_id\",\"count\",\"prefLabel\",\"type
        \",\"image_uri\")).by(__.valueMap(\"type
        \",\"probability\")).by(__.outV().valueMap
        (\"entity_id\",\"count\",\"prefLabel\",\"
        type\",\"image_uri\")),__.project(\"
        sourceProps\").by(__.valueMap(\"entity_id
        \",\"count\",\"prefLabel\",\"type\",\"
        image_uri\"))
)
```



Hundreds of millions of vertices.

- Middle vertex: Found via (graph_id, entity_id), e.g., iPhone.
- Other vertices are connected to the middle vertex via edges; "weight" on edges expressed in the "probability" property.

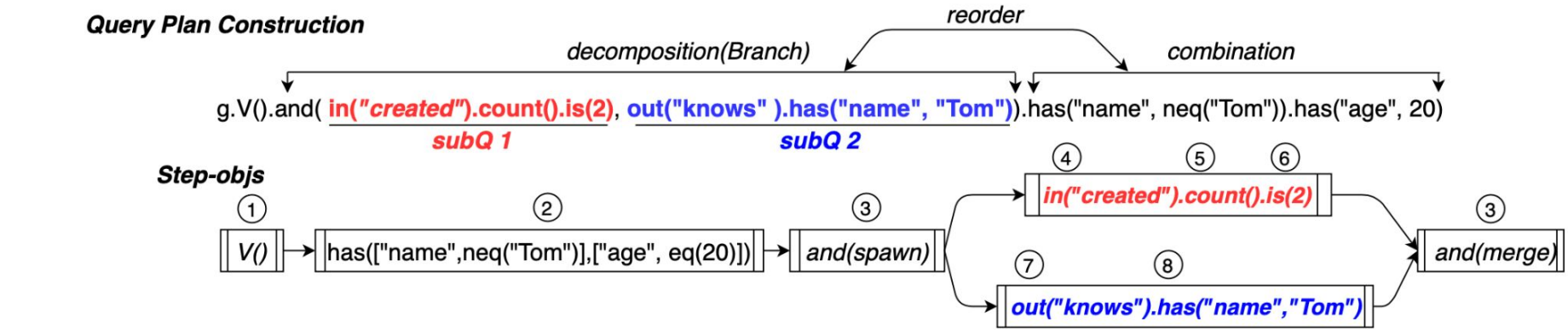
Gremlin Steps

A Gremlin query consists of a sequence of traversals. A step is a fundamental unit of computation within a traversal. Five general Steps:

1. Map: Transform step, values(), properties(), id(), label(), count(), project(), match()
2. flatMap: Transform step, out(), in(), both(), inE(), outE(), bothE(), unfold()
3. Filter: has(), is(), or(), haslabel()
4. sideEffect: store(), aggregate()
5. Branch: local(), choose(), coalesce(), union()

At times Map and flatMap are grouped together, resulting in four categories of steps.

Gremlin Query DAG



Gremlin vs. SQL

- Gremlin is a procedural/traversal query language ('how' and 'what'). SQL is a declarative/non-procedural query language ('what' only).
  - Gremlin is procedural because the user must specify a chain of steps that describes a path through the Graph DB.
- Gremlin is streaming. SQL is set oriented.
- Gremlin processes vertices and edges with properties. SQL processes tables with column headings.
  - Relationships are explicit in the Gremlin query language. With SQL, relationships are calculated by joining tables together.

Grasper: Architecture

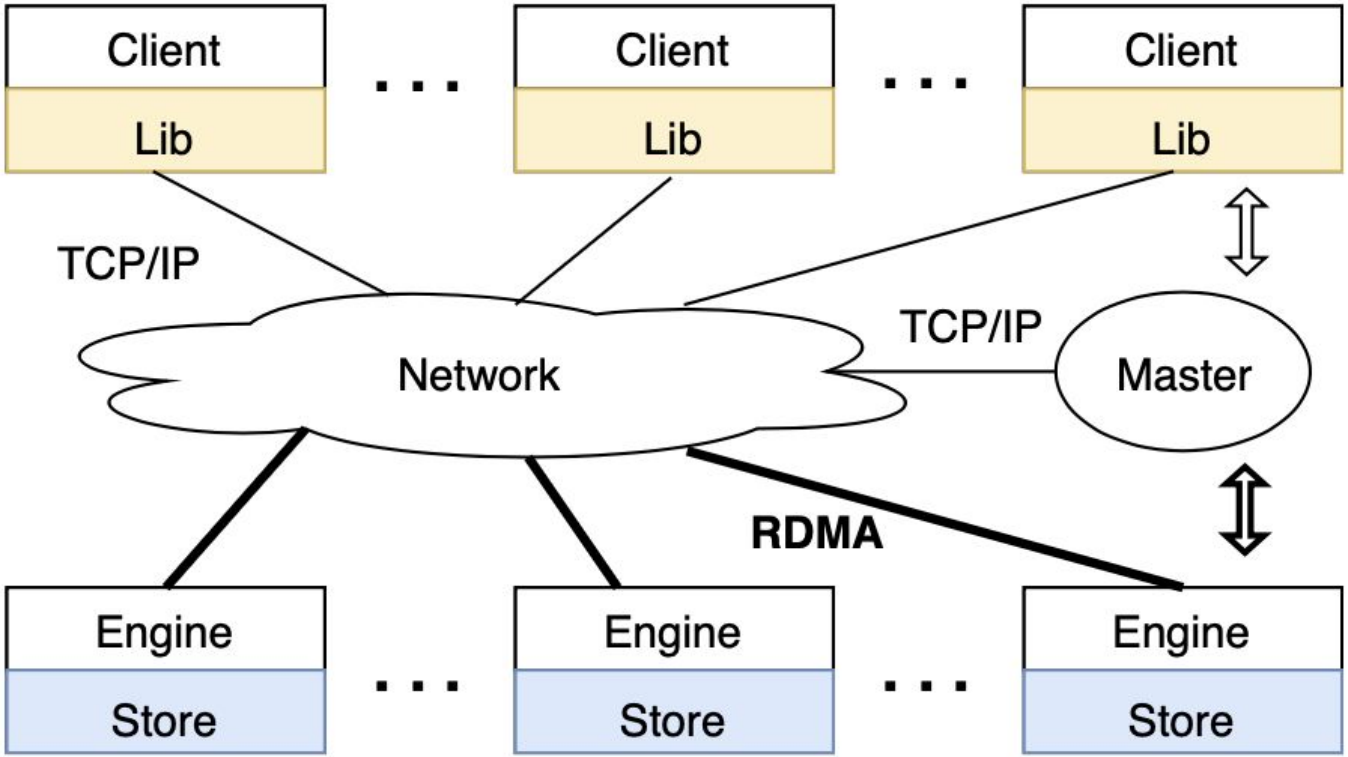
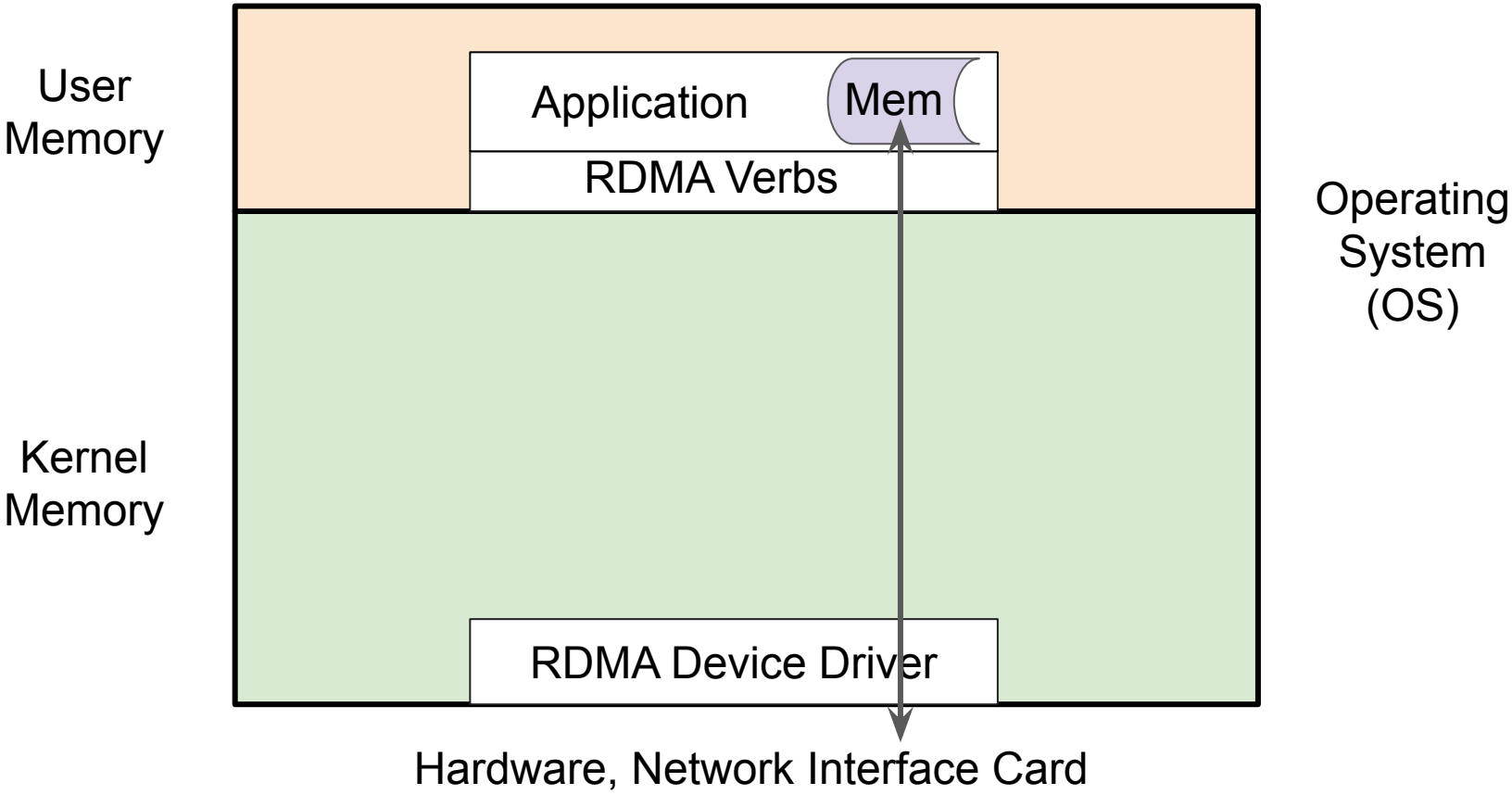


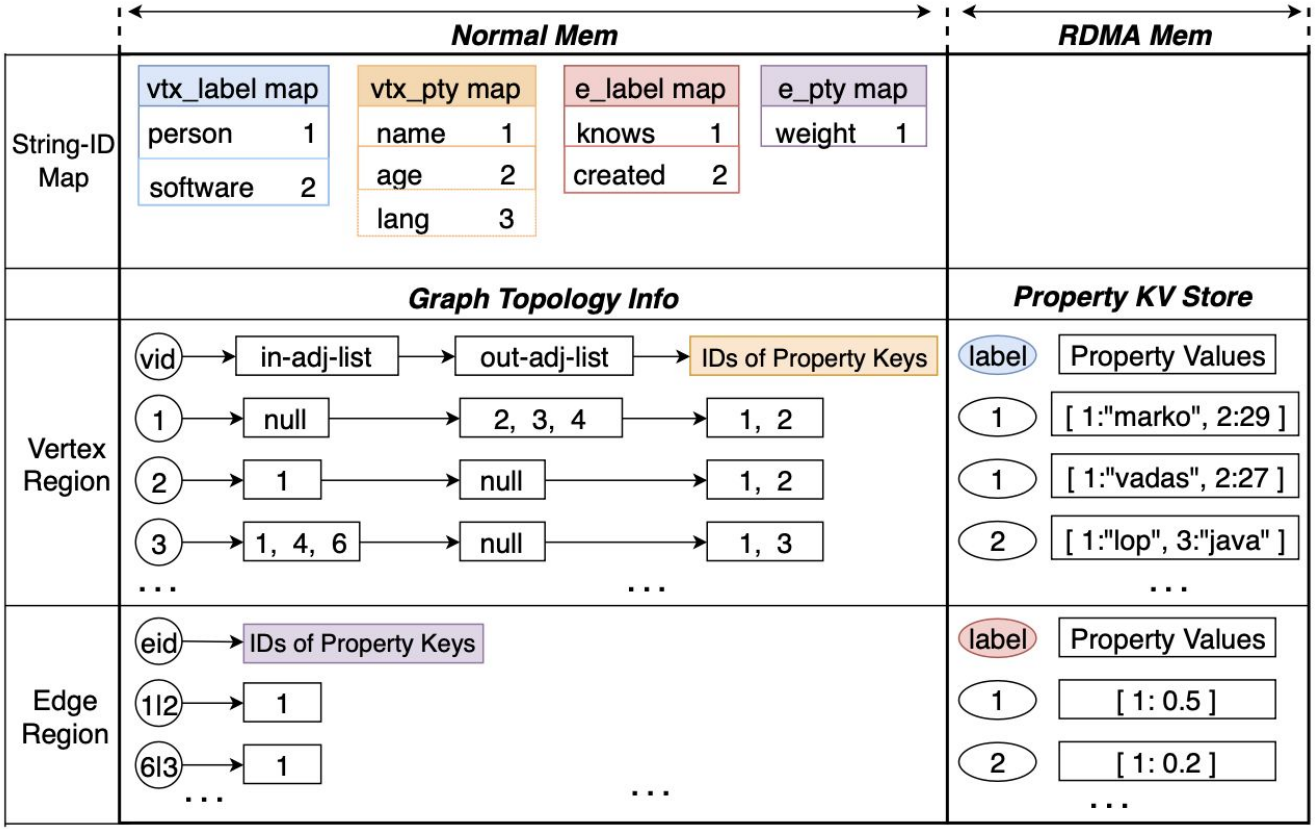
Figure 4: The architecture of Grasper

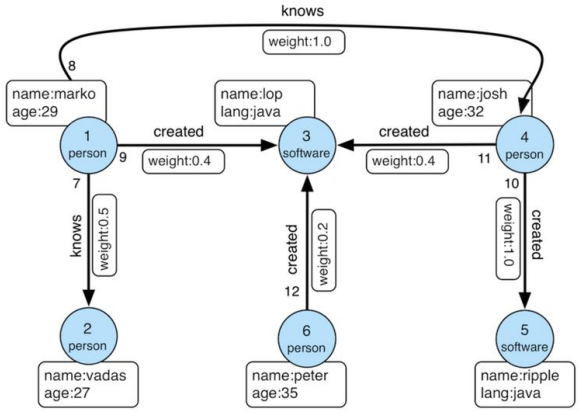
Grasper Server: Normal Memory & RDMA Memory

Engine
Store

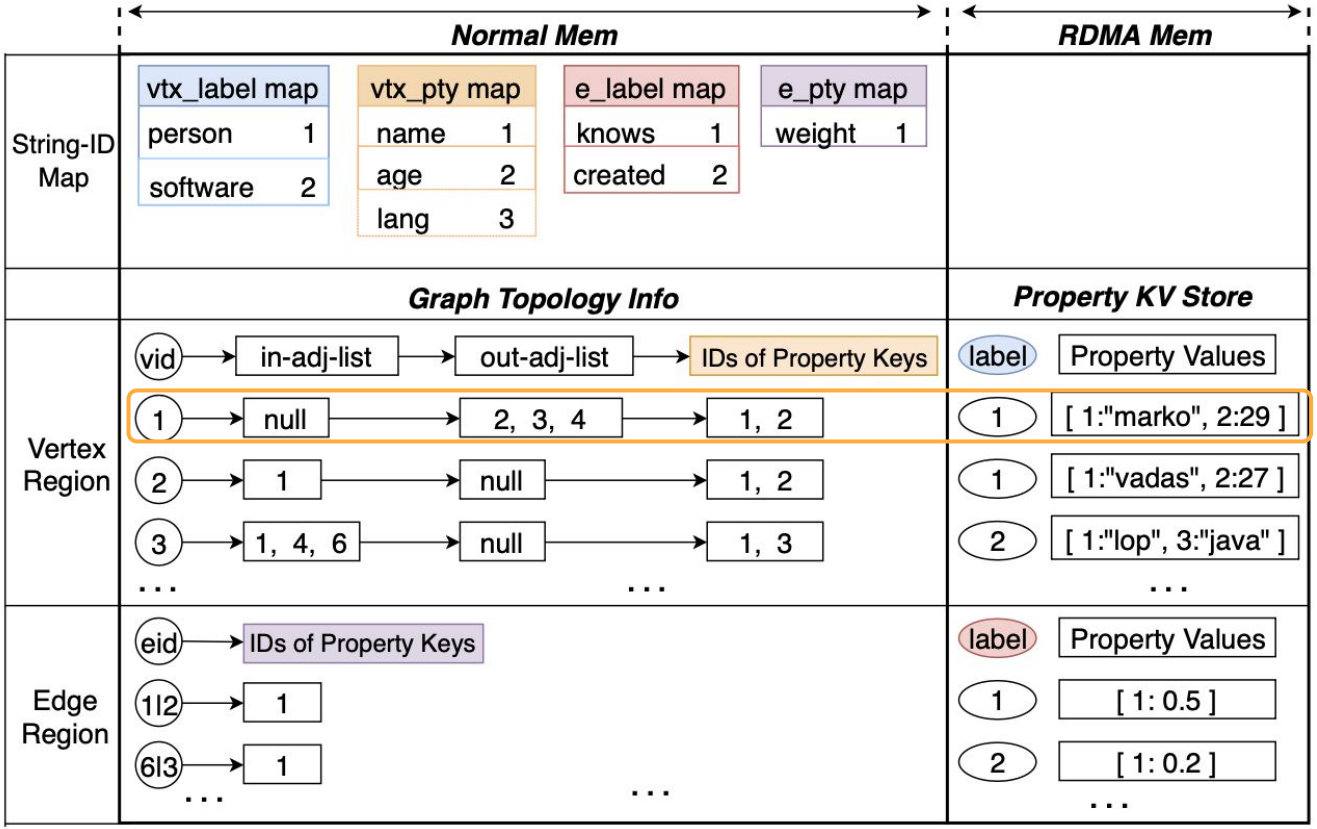


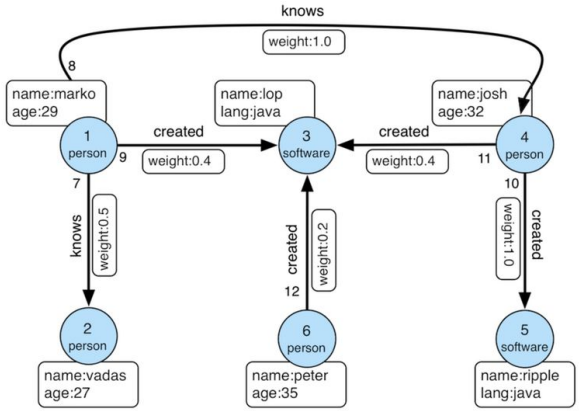
Grasper: Physical Design of Data



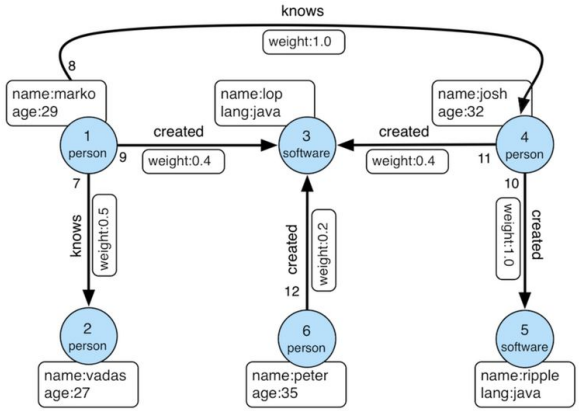


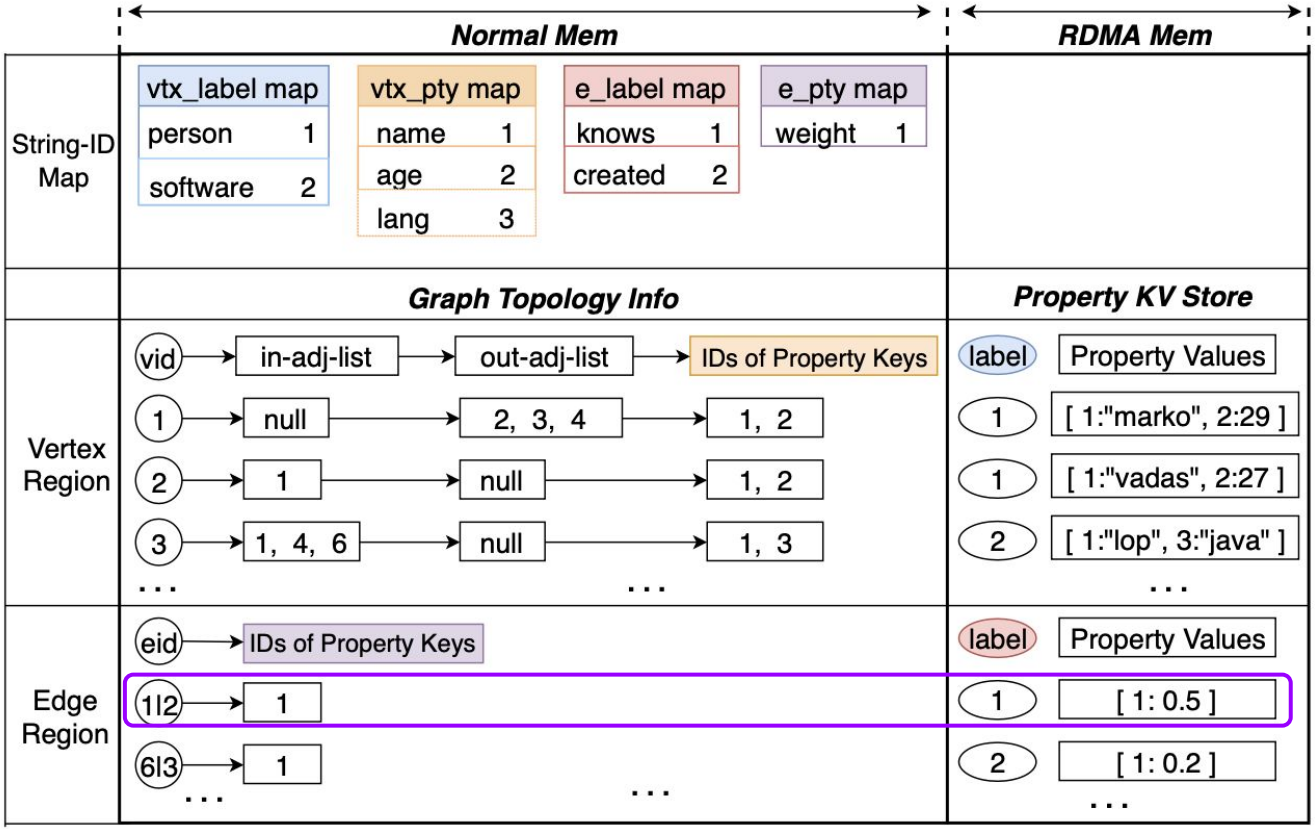
Order Vertices using their vid



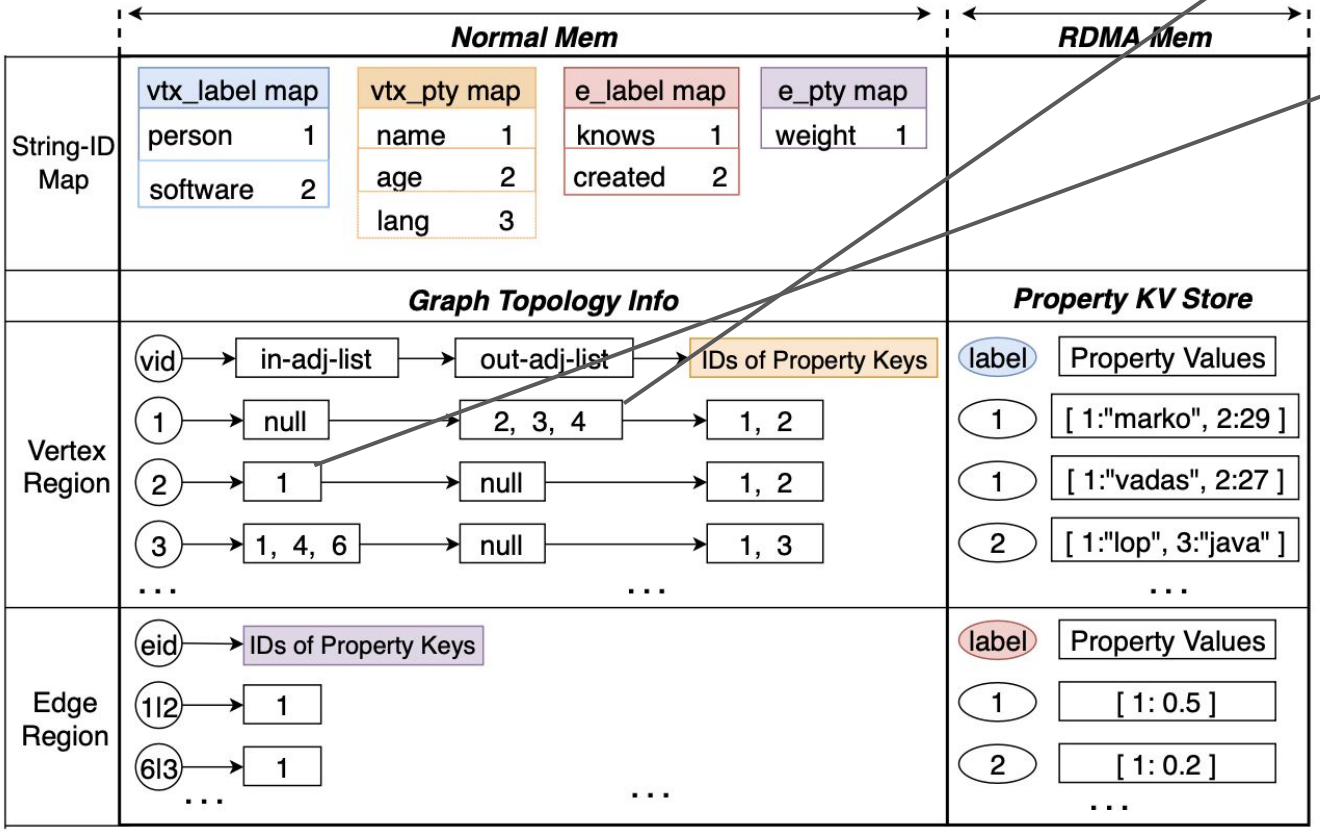


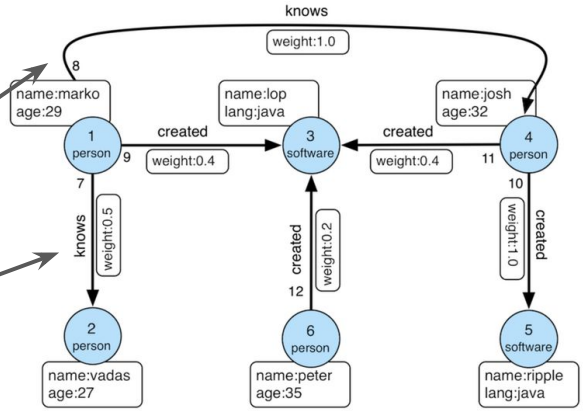
Edges Ordered



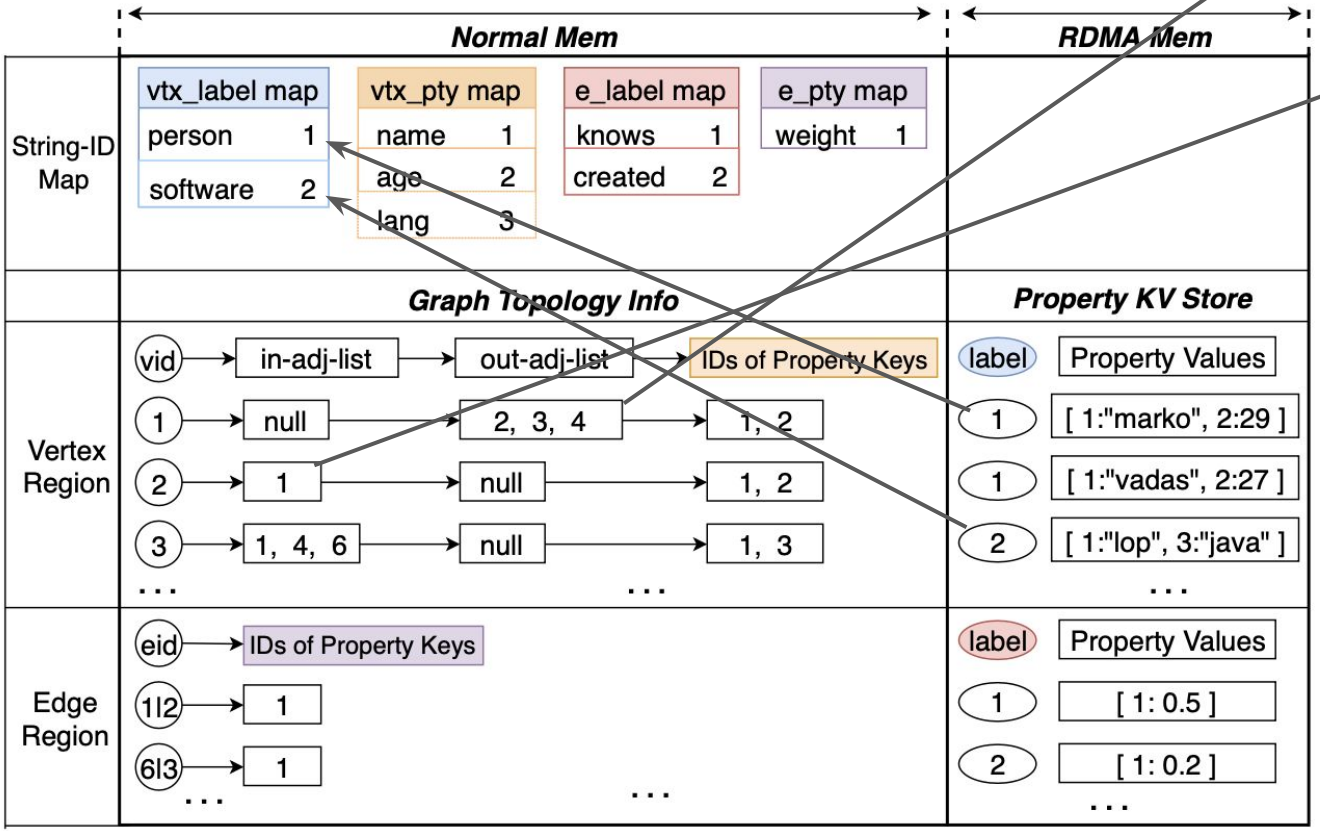


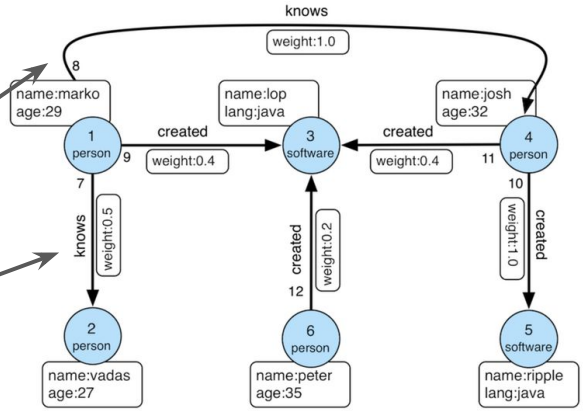
In/Out Adjacency List



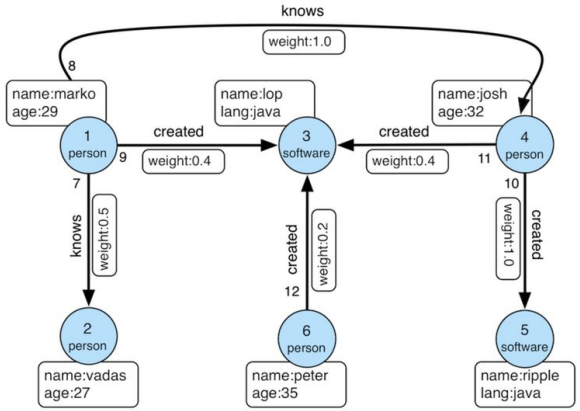


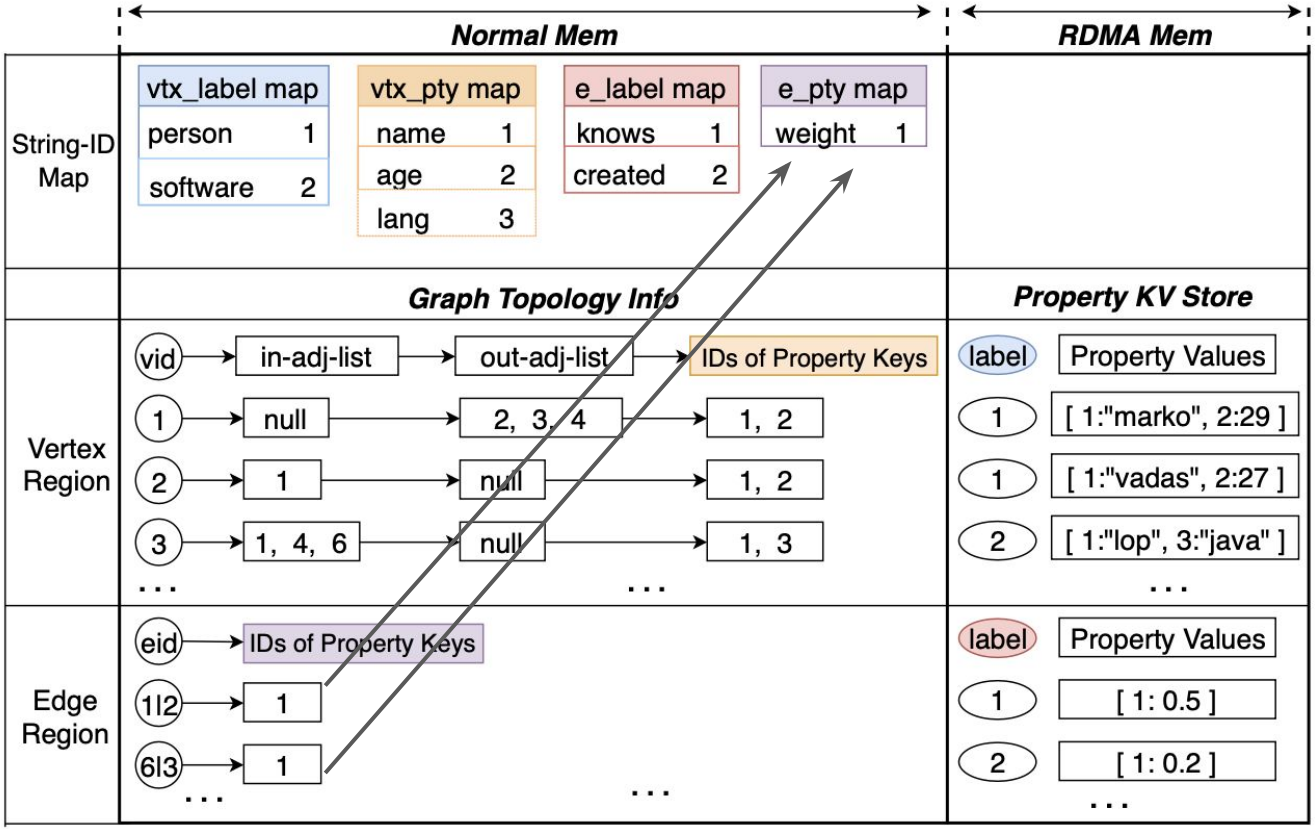
IDs for Labels: Compact Representation



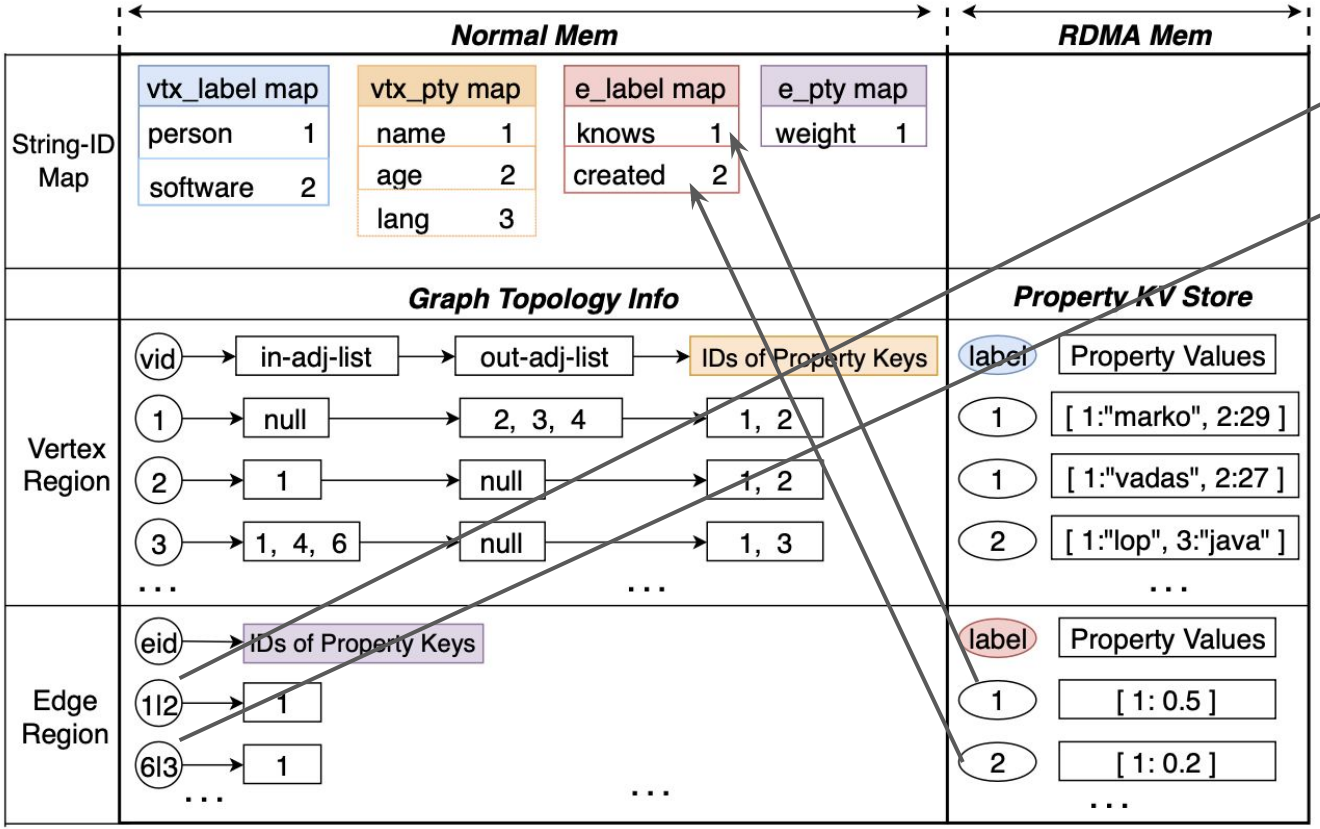


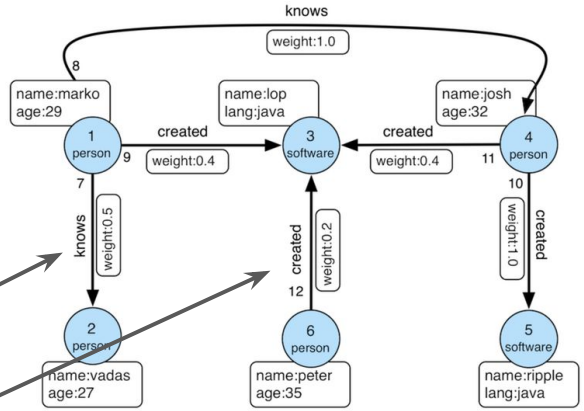
IDs for Properties



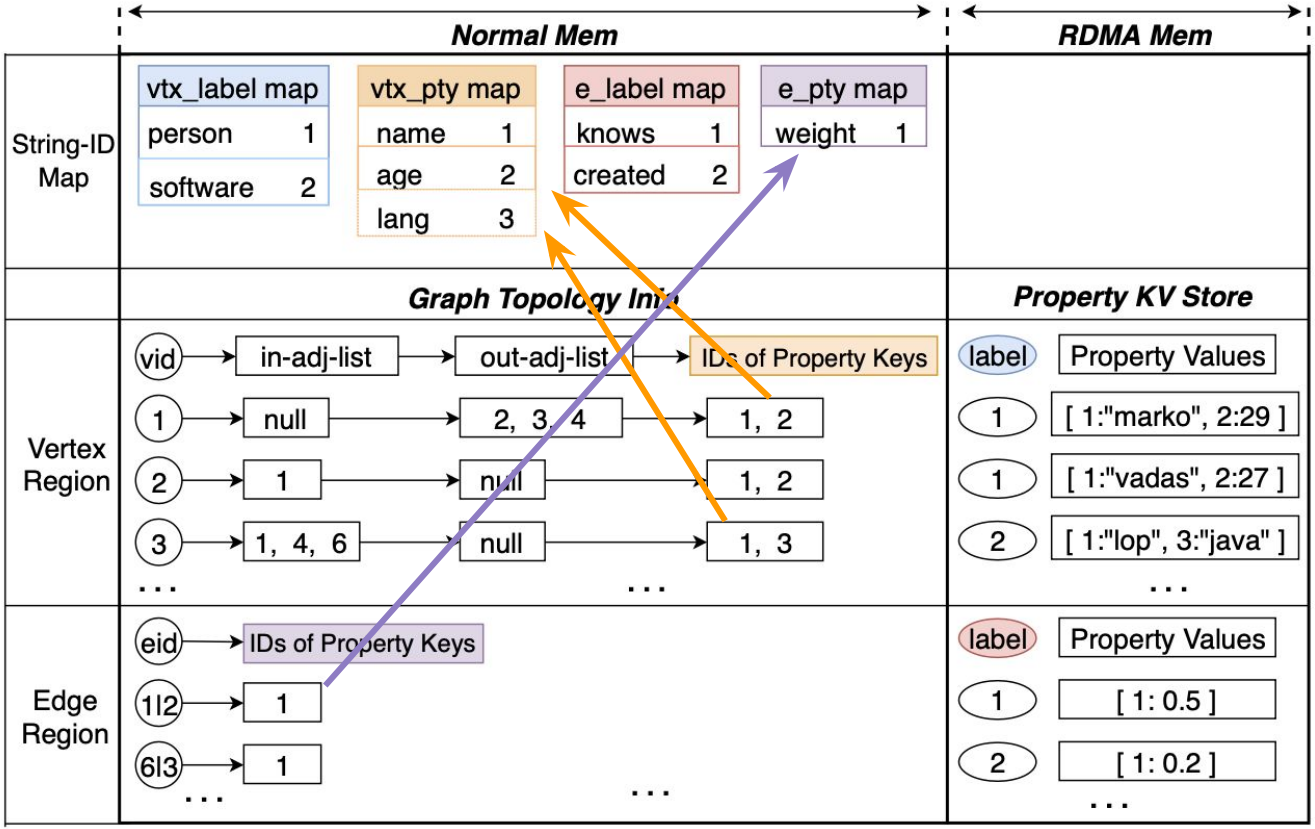


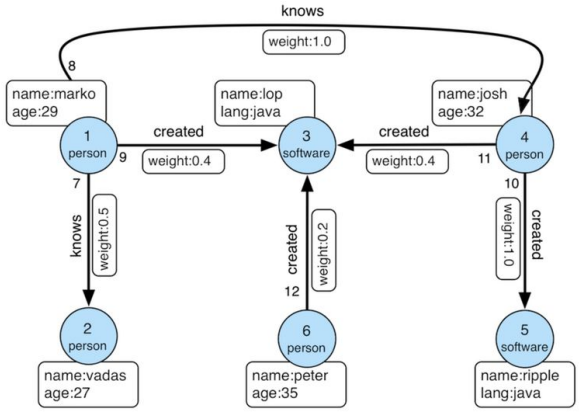
Edges


Normal Mem
RDMA Mem
String-ID Map
vtx_label map
person 1
software 2
vtx_pty map
name 1
age 2
lang 3
e_label map
knows 1
created 2
e_pty map
weight 1
Graph Topology Info
Property KV Store
Vertex Region
vid
in-adj-list
out-adj-list
IDs of Property Keys
1 null 2, 3, 4 1, 2
2 1 null 1, 2
3 1, 4, 6 null 1, 3
label
Property Values
1 [1:"marko", 2:29]
1 [1:"vadas", 2:27]
2 [1:"lop", 3:"java"]
Edge Region
eid
IDs of Property Keys
1|2 1
6|3 1
label
Property Values
1 [1: 0.5]
2 [1: 0.2]


knows
weight:1.0
8
name:marko
age:29
1
person
created
9
weight:0.4
3
software
name:lop
lang:java
created
weight:0.4
11
4
person
name:josh
age:32
7
knows
weight:0.5
2
person
name:vadas
age:27
12
created
weight:0.2
6
person
name:peter
age:35
10
weight:1.0
created
5
software
name:ripple
lang:java

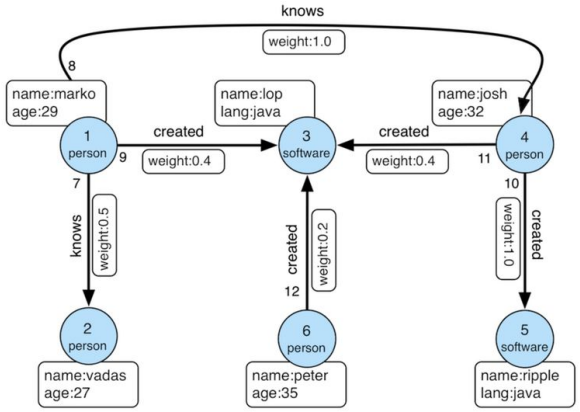
IDs for Properties

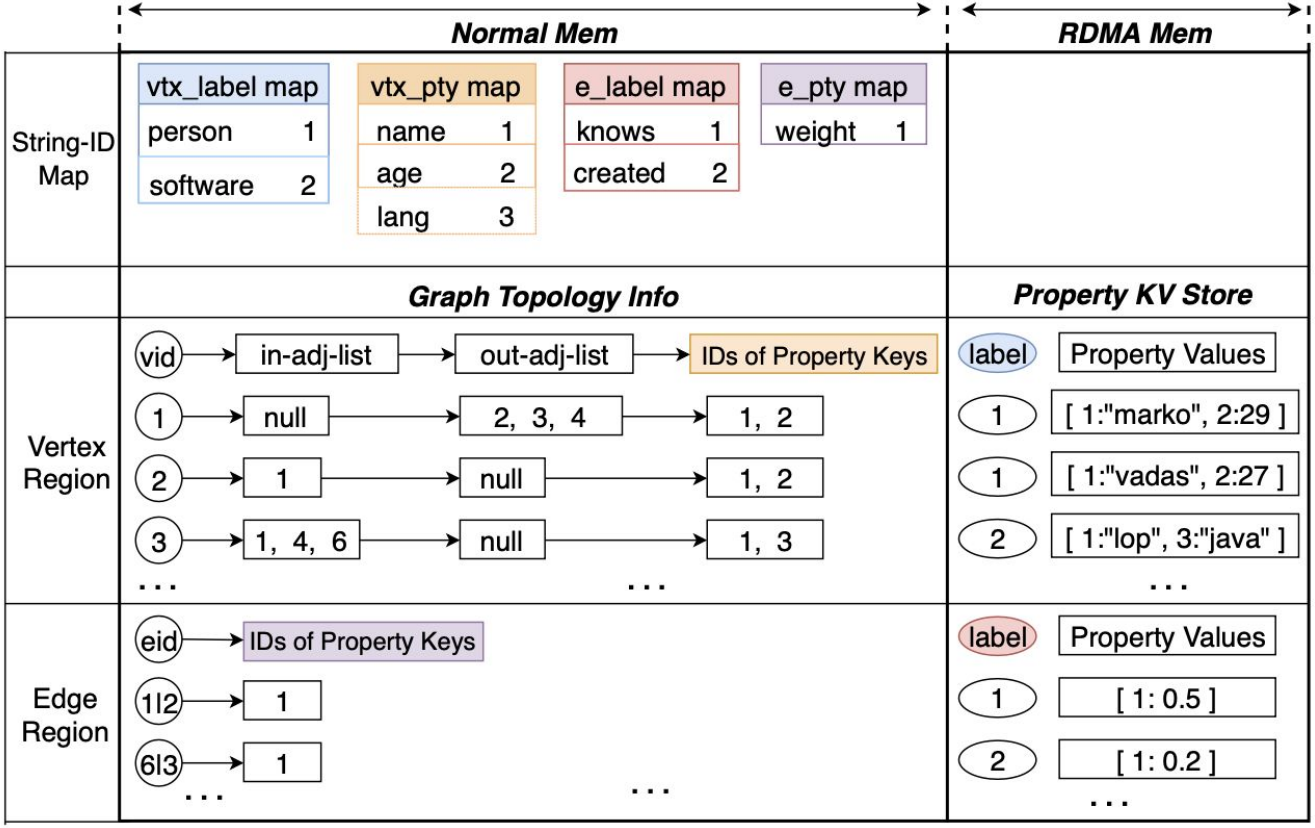




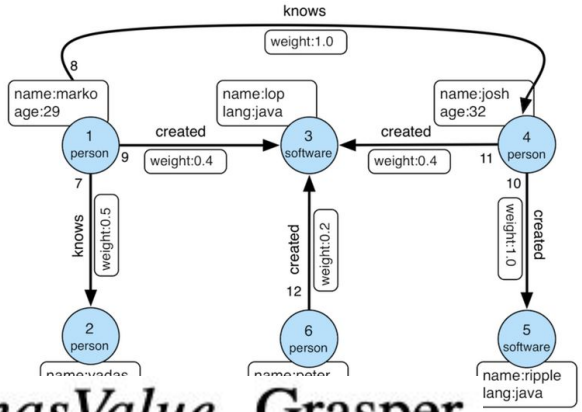
Questions

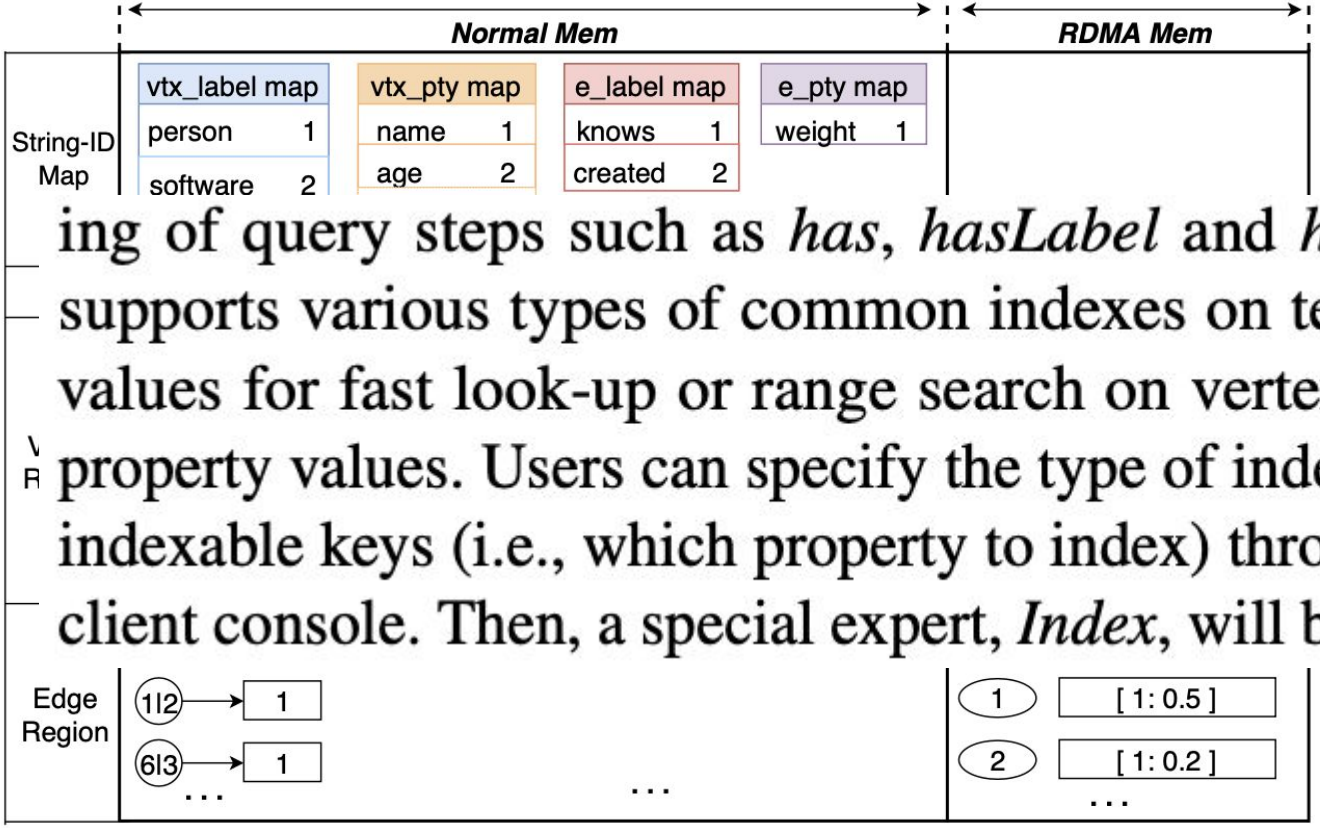
How to find an edge between 2 nodes?





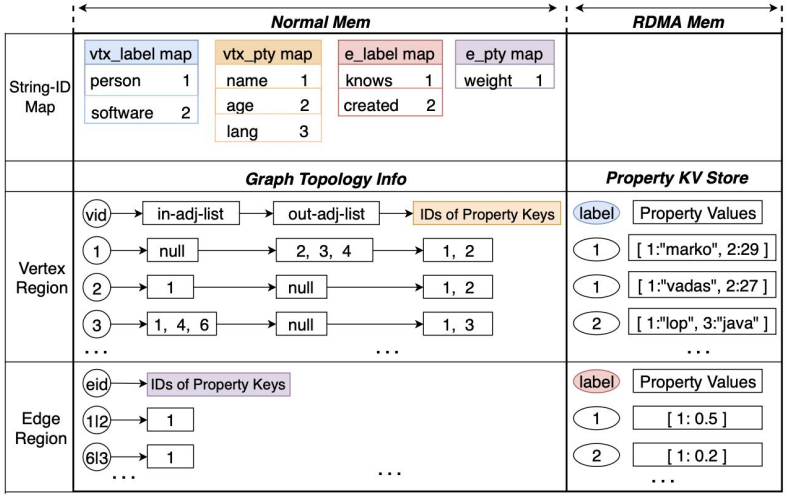
How to find an edge between 2 nodes?





ing of query steps such as *has*, *hasLabel* and *hasValue*. Grasper supports various types of common indexes on text and numerical values for fast look-up or range search on vertex/edge labels and property values. Users can specify the type of index to build and the indexable keys (i.e., which property to index) through an interactive client console. Then, a special expert, *Index*, will be invoked on each

Fast RDMA Network is a MUST!



because even the simplest query needs to frequently access the properties on remote servers through the network. We remark that the per-

Grasper: Architecture

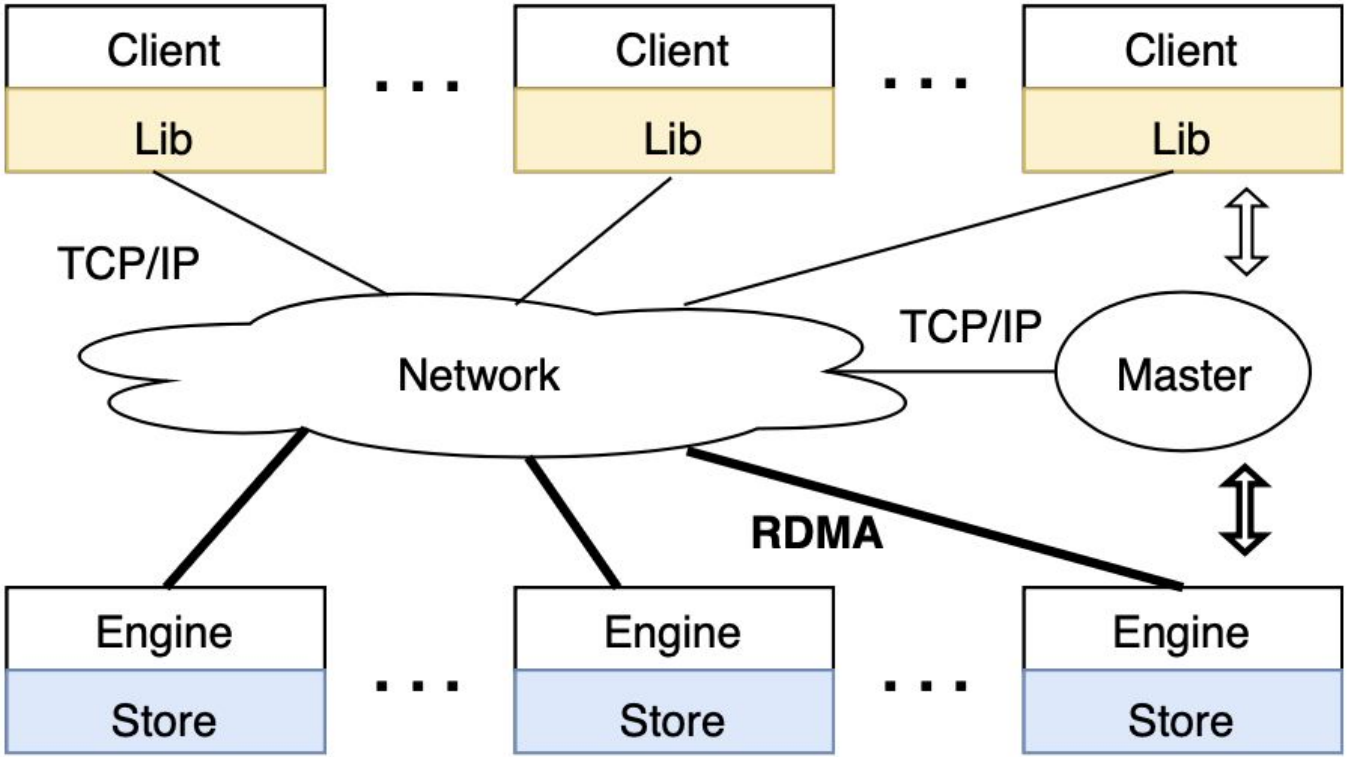


Figure 4: The architecture of Grasper

Grasper: Data Store & Query Engine

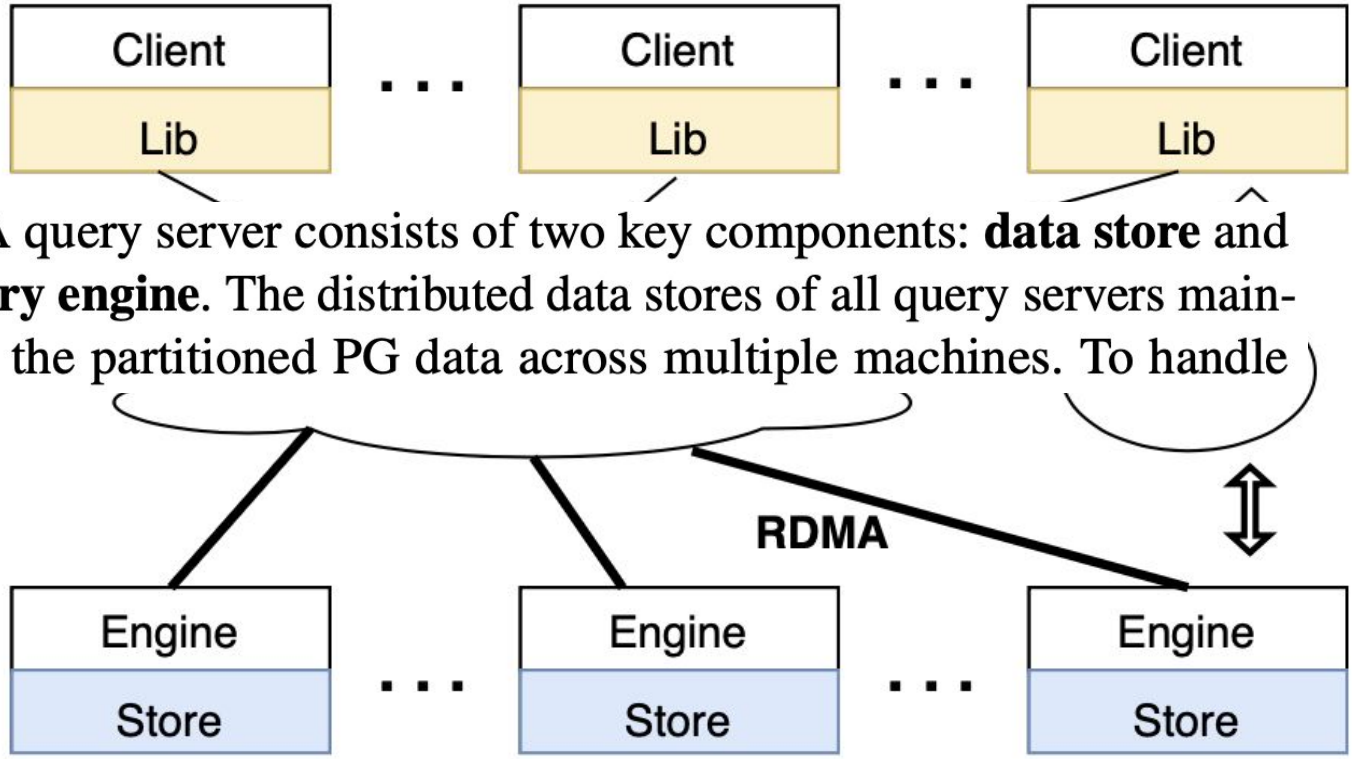


Figure 4: The architecture of Grasper

Grasper: Master

Grasper also assigns a master node to monitor the progress of query execution on each query engine through a heartbeat mechanism and accordingly to coordinate the assignment of incoming queries from clients to servers based on the current load of each engine.

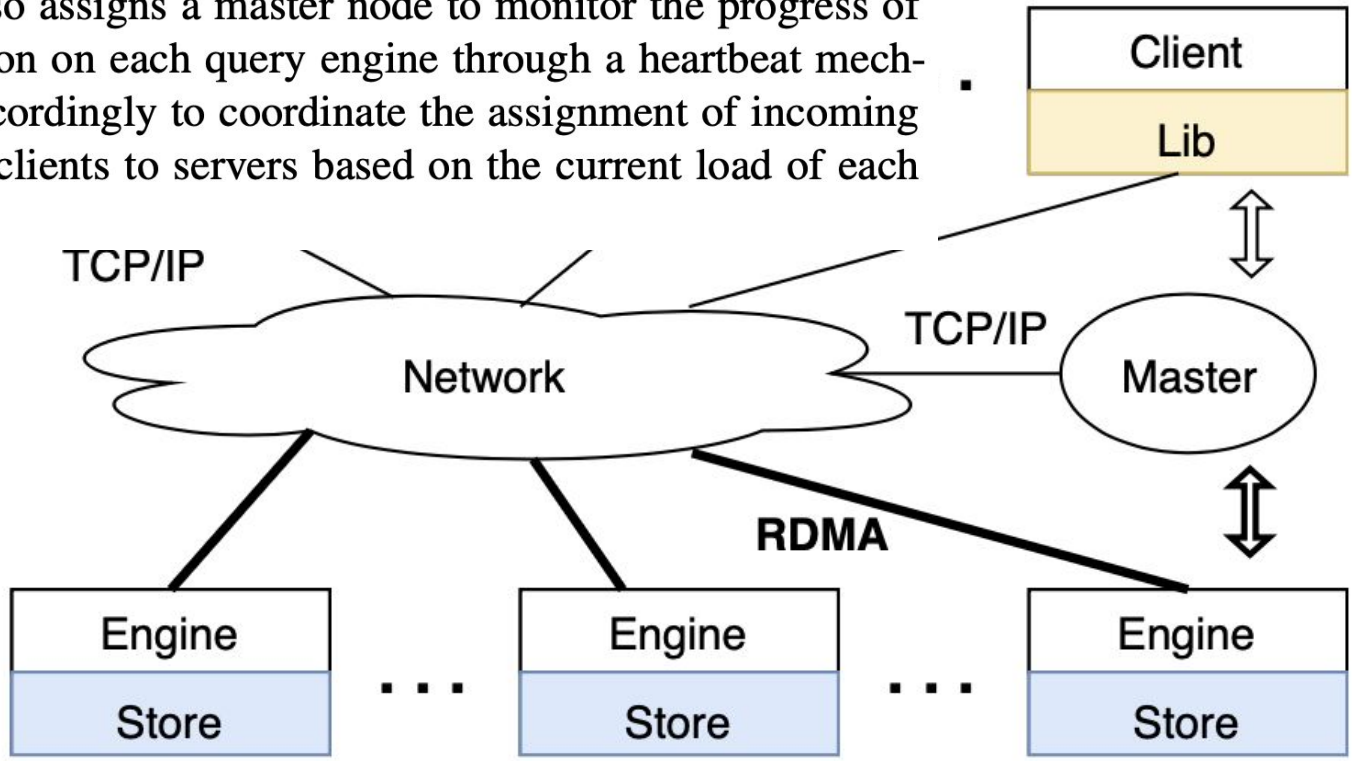
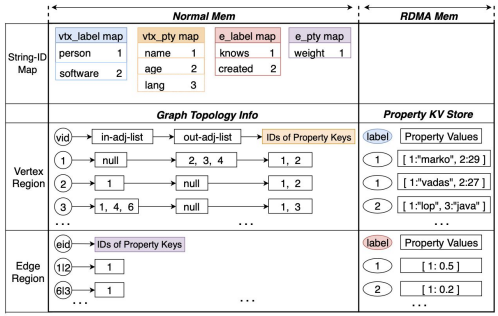


Figure 4: The architecture of Grasper

Partition Memory of Each Node into Two

- Normal Memory: Graph topology accessed during query execution and intermediate results generated by a query.
- RDMA Memory: Graph data such as vertex/edge labels, properties and their values. It is a distributed key-value store.
- Grasper loads the PG data from HDFS. Vertices/edges/labels are hash partitioned across the normal and RDMA memory of nodes. Why?
  - More network communication for processing query is not an issue because of the fast RDMA.
  - Co-locating graph topology and properties togethers results in unbalanced data and load.
  - A balanced distribution of data → balanced CPU utilization.
- What is the consequence of the above?

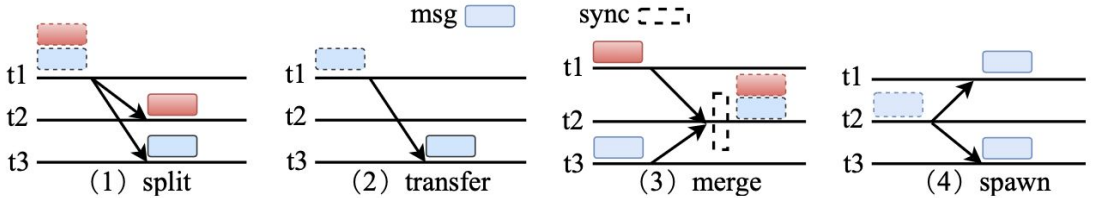


HDFS: Hadoop Distributed File System

- Inspired by the Google File System, GFS.
- Design objectives: High throughput for sequential access to data.
- Master-slave architecture:
  - Master is a NameNode
    - Stores all the metadata such as file names, directory structure, file permissions, and the location of each data block on data nodes.
    - Manages the file system namespace and regulates client access to files.
    - 1 Master per cluster.
  - Slave is a DataNodes
    - Nodes that store the content of files.
    - Deployed using commodity hardware in a cluster.
    - Data is partitioned into fixed-size chucks called blocks with default size of 128 MB or 256 MB.
    - A slave may create a block, delete a block, replicate a block in response to an instruction from the Master/NameNode.
    - A client may access a slave to read a block of a file.

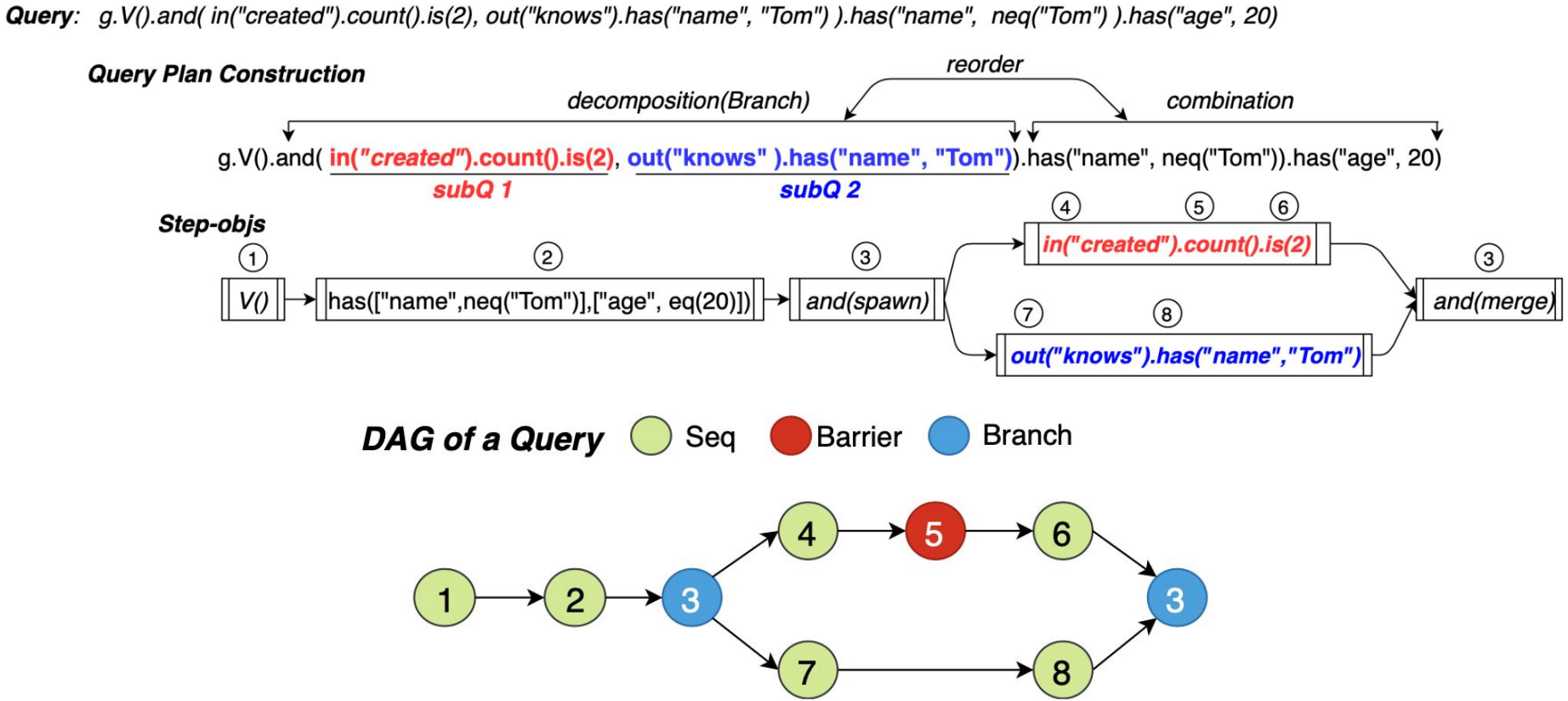
Query Engine: Concepts

- Data flow execution engine: A message is a piece of intermediate or final query result generated from a specific query step. It has metadata.
  - There is a fixed maximum size on a message. If a message becomes larger than the maximu, it is split into multiple messages.



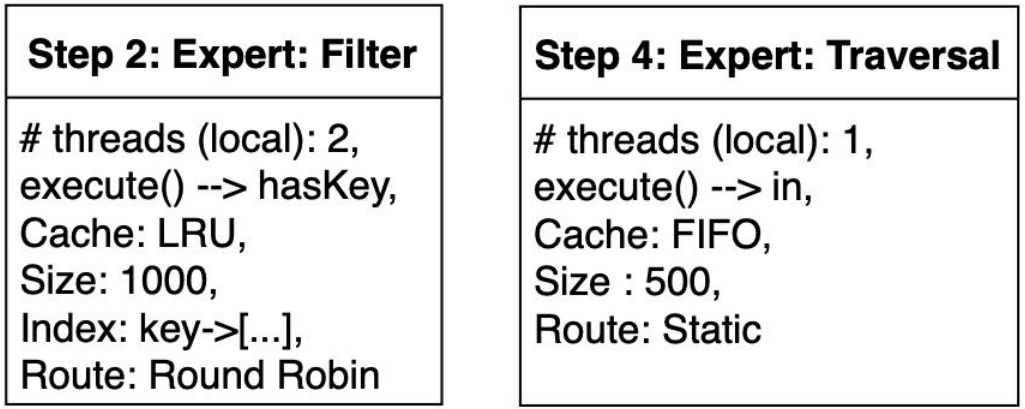
- Flow Type: The flow pattern of each query step according to its functionality. Three types of flow:
  - Sequential: A sequential query step (e.g., in, out, has) receives a message, processes it, and generates new message(s) to be proceessed by the next step.
  - Barrier: Collects messages from the previous step and performs a functional aggregation the collected data prior to proceeding to the next step, e.g., max, count.
  - Branch: Creates multiple sub-query braches, where the message received by the step is duplicated as the input of the sub-queries for parallel processing.

Gremlin Query DAG



Observation 1:

- There are groups of Gremlin steps that provide similar functionalities, e.g., filter-based steps such as *has(), hasKey(), and hasValue()*.
- Assumption 1: These groups of steps have the same access pattern.
- Design decision 1: Apply tailored optimizations on different groups. An optimization may use a specific caching strategy, cache size, or message routing rule.



Query Engine: Operator/Expert

- A query operator or expert is responsible for processing one specific category of query steps that share similar functionalities.
- An expert may use multiple threads to process the query steps of multiple queries with shared optimizations.
- An expert is aware of the hash partitioning of data. It fetches its referenced data from the appropriate fragment/Engine for processing.
- An expert may cache data from another Engine. Its replacement policy is specific to how it references data, e.g., LRU for *Filter* and FIFO for *Traversal*.

Table 1: The expert pool in Grasper

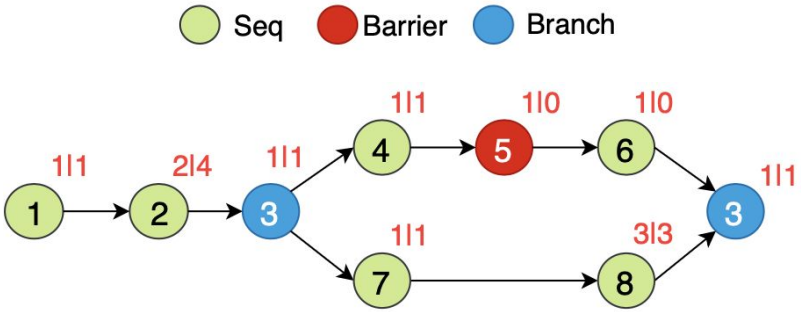
Expert	Query Steps
Init	g.V(), g.E()
End	N/A [to aggregate the final results]
Traversal	in, out, both, inE, outE, bothE, inV, outV, bothV
Filter	has, hasNot, hasKey, hasValue
Range	range, limit, skip
Order	order
...	...
Group	group, groupCount
Math	min, max, mean,
BranchFilter	and, or, not

Table 2: Thread regions with core binding

Thread Region	Experts
Cache_Seq	Filter, FilterLabel, Key, Values, Properties, Label
Cache_Barrier	Group, Order
Traversal	Traversal, Init
Normal_Seq	Is, As, Select, Where
Normal_Barrier	Count, Range, Math, Aggregate, Cap, Dedup
Normal_Branch	Branch, BranchFilter

Adaptive Parallelism

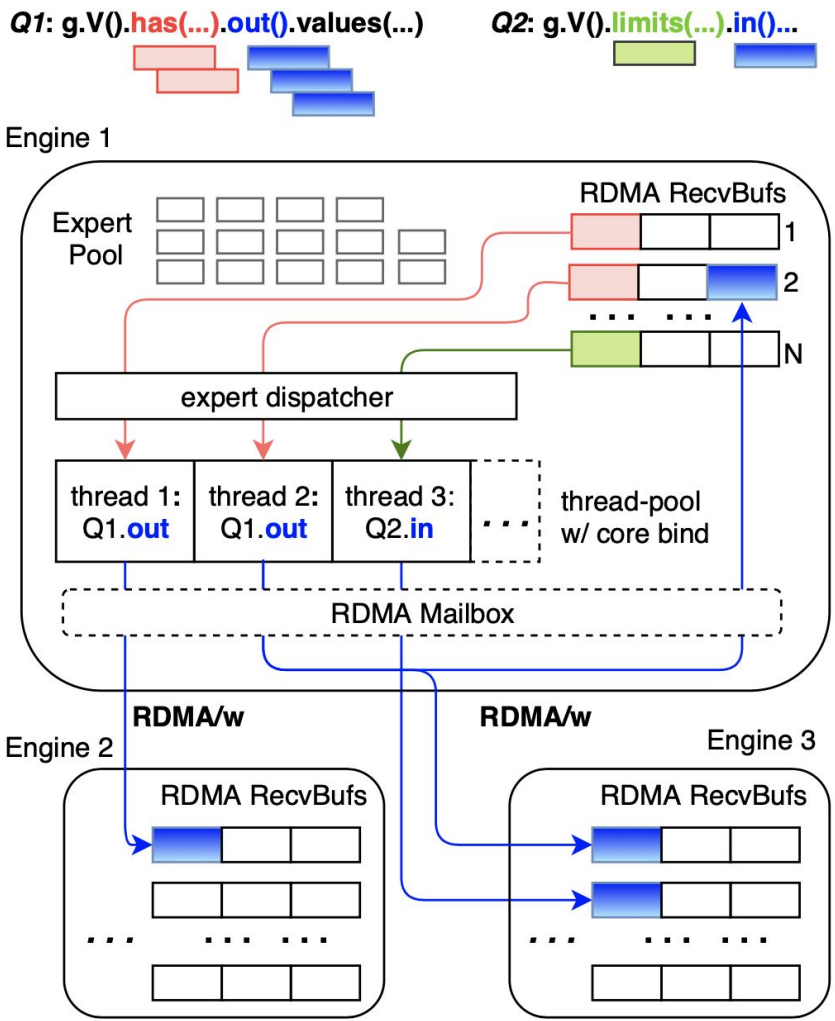
- Workload of a query step is quantified using the number of messages it receives at each query engine → an expert may set the degree of parallelism to the number of messages.
- The degree of parallelism may be specified in the query plan. The following assumes a deployment with 2 query engines, i.e., Engine 1 and Engine 2.



"2|4" on step 2 means that engine 1 and engine 2 assign 2 and 4 threads to process the *hasKey* operator in parallel. This also indicates that the load of *hasKey* on engine 2 is heavier than that on engine 1. Note that the parallelism of the *barrier*-type step 5 is 1 globally because of its aggregate function. In constrast, both *sequential* and *branch* query steps have no limitation on parallelism.

Architecture

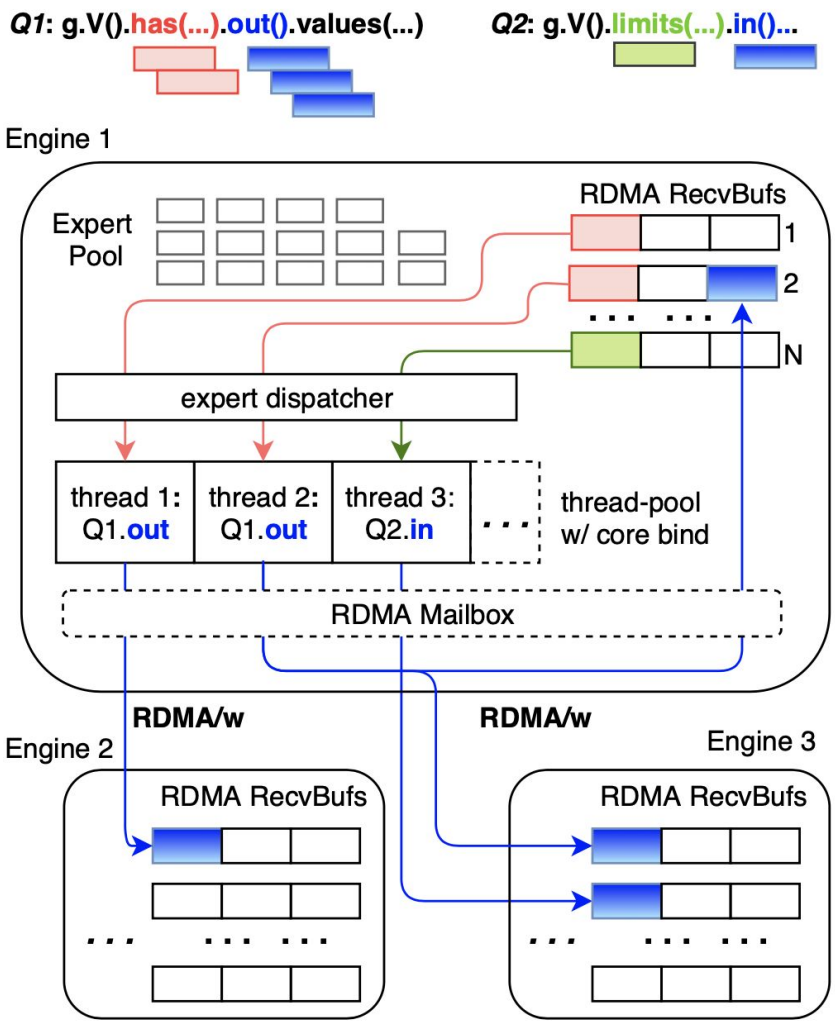
- When a query engine is launched, its expert pool will be initialized and all experts will be constructed and kept alive until the engine shuts down.
- A thread is bound to a core to avoid the overhead of context switching.
- The number of cores in an Engine dictates the overall number of threads.
- Each thread has its own RDMA RecvBufs.
- An Engine may execute multiple queries concurrently.
- Threads of an Expert share the cached and indexed data of the expert.



Architecture Illustrated

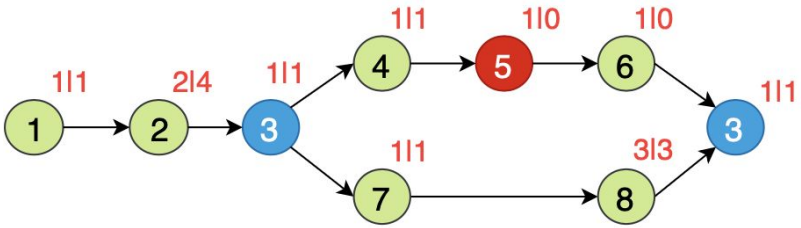
Illustration of concepts:

trate using Q1 and Q2 in Figure 11. The *has*() step of Q1 generates 2 messages (colored in pink) and the *limits*() step of Q2 generates 1 message (colored in green). Then, all these 3 messages are processed in parallel by 3 threads, i.e., threads 1-2 (working on Q1.*out*) and thread 3 (working on Q2.*in*). These 3 threads are employed by the *Traversal* expert with shared cache and indexes as well as other specific optimizations. That is, the cached data, indexed data (if any), and other data structures of the *Traversal* expert are being utilized by these threads through internal sharing. Accordingly, the CPU utilization and memory locality can be both improved. To distribute the newly generated messages (colored in blue) evenly among threads in the receiving query engines, the *Traversal* expert applies its own routing rule to determine the specific location (i.e., thread and engines) of each message.

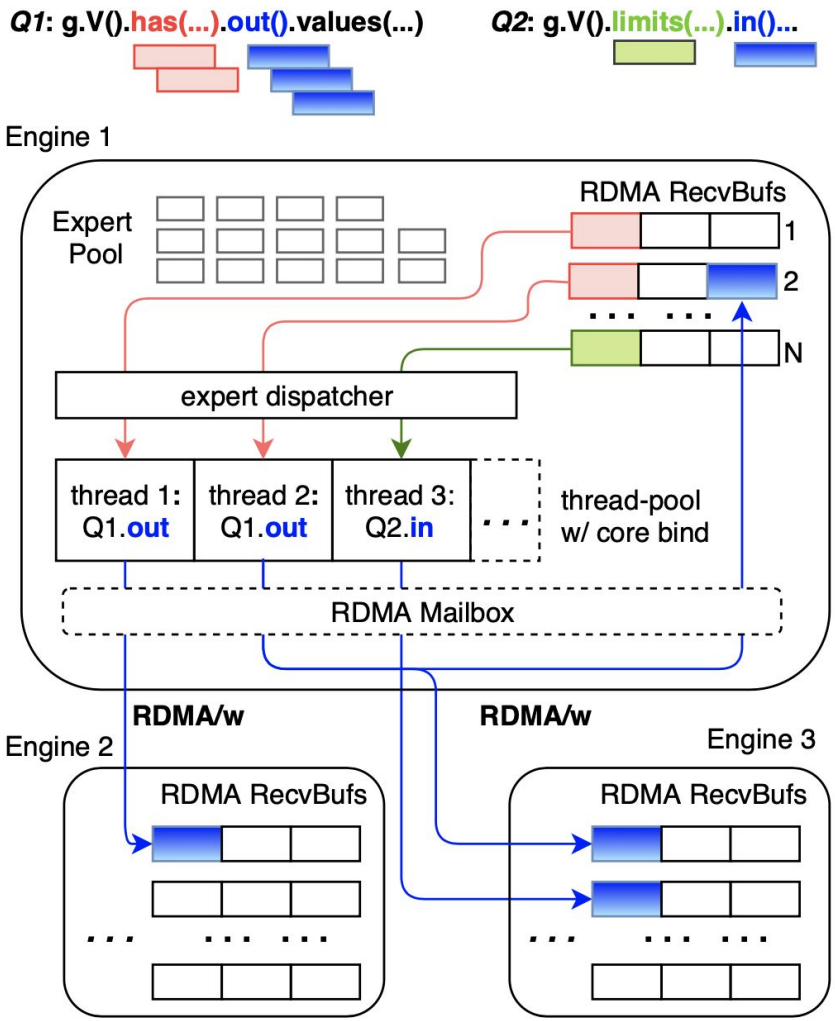


Stragglers

- A straggler is a task that takes significantly longer than the others in the same query to complete.
- Example: Operator 8 may take 10x the time required by the other operators in the following query plan:



Why are stragglers undesirable?

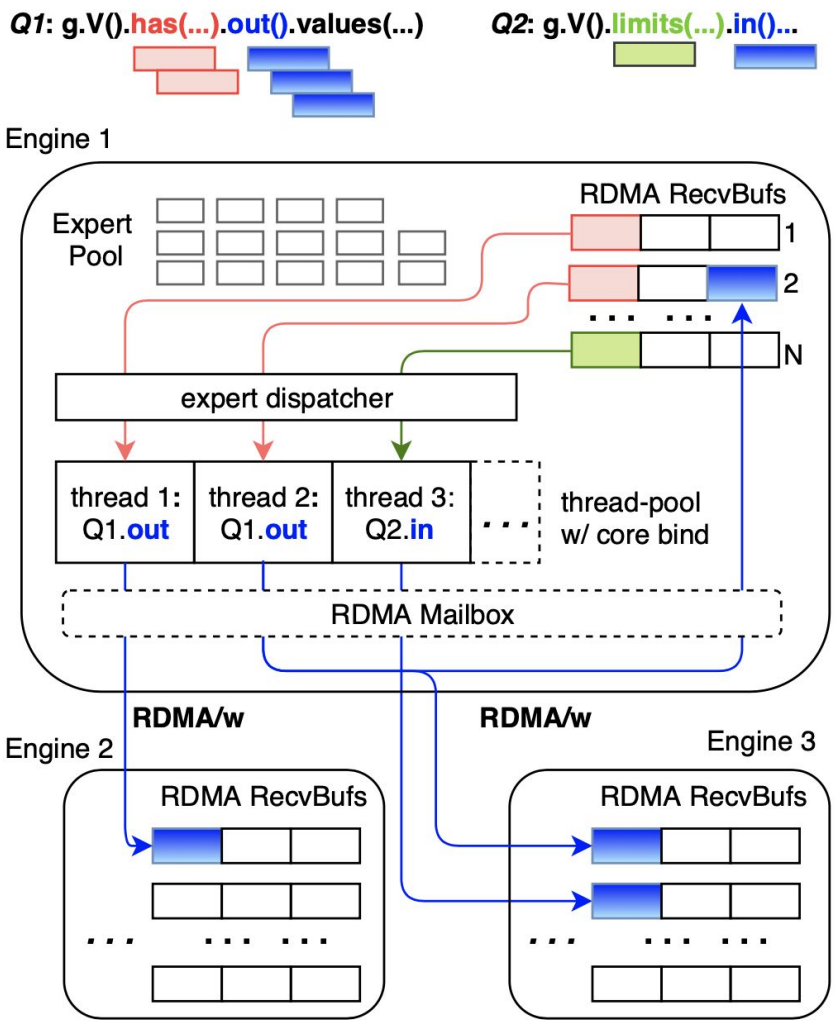


Solution: Work Stealing

- With Grasper, an idle thread steals messages from the RecvBufs of other busy threads to improve response time.
- There is a stealing priority order: An idle thread first checks the RecBufs of the other threads in the same thread region (Traversal) on the same node, and then those of the other regions (Filter) in the same node, and finally those of the other Engines (nodes) until it obtains a message to process.

Is there a race condition?

What is the complexity of thread stealing?



Expert Model

1. Adaptive parallelism control at step-level inside each query;
2. Tailored optimizations for various query steps according to their characteristics;
3. Locality-aware thread binding and load balancing.

1. Expert Model ensures that the runtime parallelism degree of an expert is adaptive according to the workload.
2. Expert Model improves the efficiency of query processing,
3. Expert Model handles the underlying cache locality and load balancing among the threads physically.

Experimental Evaluation

- Hardware platform:
  - A cluster of 10 machines/engines. Each has two 8-core Intel Xeon 2.1 GHz processors and 128 GB of memory.
  - Each machine uses a Mellanox ConnectX-3 40 Gbps RDMA card.
  - 24 computing threads per machine.
- 3 different benchmarks with 8 queries.
  - LDBC with 3 interactive complex/short queries.
    - IC1-IC4 are interactive complex
    - IS1-IS4 are interactive simple
  - AMiner with 8 shown queries:
    - AMiner is a real-world property graph.
    - Q3, Q4, and Q5 produce large intermediate results.
  - Twitter is a synthetically generated benchmark.

Table 3: Dataset statistics

Dataset	$\|V\|$	$\|E\|$	$\|VP\|$	$\|EP\|$
LDBC	59,308,744	357,617,104	321,281,654	101,529,501
AMiner	68,575,021	285,667,220	291,161,548	120,381,452
Twitter	52,579,682	1,963,262,821	320,732,961	577,955,736

Table 4: Query benchmark

Q1	g.V().has([*filter*]).properties([*property*])
Q2	g.V().hasKey([*filter1*]).hasLabel([*label*]).has([*filter2*])
Q3	g.V().has([*filter*]).in([*label*]).values([*key*]).max()
Q4	g.E().has([*filter1*]).outV().dedup().has([*filter2*]).count()
Q5	g.E().has([*filter1*]).not(outV([*label*]).has([*filter2*])) .groupCount([*key*])
Q6	g.V().has([*filter*]).and(out([*label1*]).values([*key1*]).min().is([*predicate1*]), in([*label2*]).count().is([*predicate2*])).values([*key2*])
Q7	g.V().has([*filter1*]).as('a').union(out([*label1*]), out([*label2*]).out([*label3*])).in([*label4*]).where(neq('a')).has([*filter2*]) .order([*property*]).limit([*number*])
Q8	g.V().has([*filter1*]).aggregate('a').in([*label1*]).out([*label2*]). .has([*filter2*]).where(without('a'))

Ablation Study: Contribution of a Design Decision

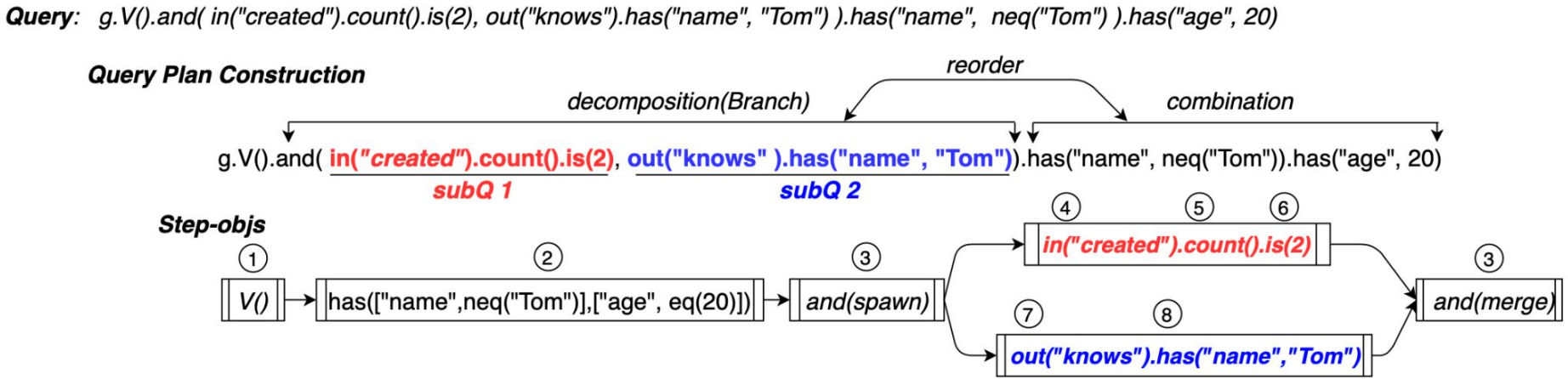
- RDMA
- Query Optimization
- Work Stealing

Table 8: Query latency (in *msec*) of [Grasper-X] (using 10 machines)

LDBC	IC1	IC2	IC3	IC4	IS1	IS2	IS3	IS4
Grasper	271	16.7	388	77.3	0.30	2.19	0.91	0.32
-RDMA	1349	17.97	1253	260	1.04	2.57	2.06	1.26
-Q.Opts	374	19.39	558	81.26	0.31	2.38	0.93	0.32
-Steal	488	24.68	671	127	0.57	3.25	1.31	0.54
AMiner	**Q1**	**Q2**	**Q3**	**Q4**	**Q5**	**Q6**	**Q7**	**Q8**
Grasper	0.17	0.42	17.3	45.2	104	28.8	2.32	4.41
-RDMA	0.54	1.18	21.54	70.47	222	30.69	9.09	6.23
-Q.Opts	0.17	4254	22.84	417	131	35.49	2.89	4.34
-Steal	0.23	0.61	20.91	57.62	111	33.44	4.01	6.07

Query Optimization Techniques

- Decomposition: Multiple parallel sub-queries for branch-type *and*, *or*, *union*.
- Combination: Combine filter-based steps on vertices/edges/properties into one.
- Reordering: Move steps with constraints to the front to reduce intermediate results.



Scalability is sublinear

- Compare 1 node with 10 node configuration: Ideal speedup should be 10x.

Table 6: Query latency (in *msec*) of single-machine systems

LDBC	IC1	IC2	IC3	IC4	IS1	IS2	IS3	IS4
Grasper	1935	75.1	2550	223	0.48	2.51	1.38	0.13

Table 8: Query latency (in *msec*) of [Grasper-X] (using 10 machines)

LDBC	IC1	IC2	IC3	IC4	IS1	IS2	IS3	IS4
Grasper	271	16.7	388	77.3	0.30	2.19	0.91	0.32

Speedup:	7x	5x	7x	3x	2x	7x	2x	0.4x

Grasper in an in-memory system

- With the Twitter data, it runs out of memory with a cluster consisting of 2 nodes.

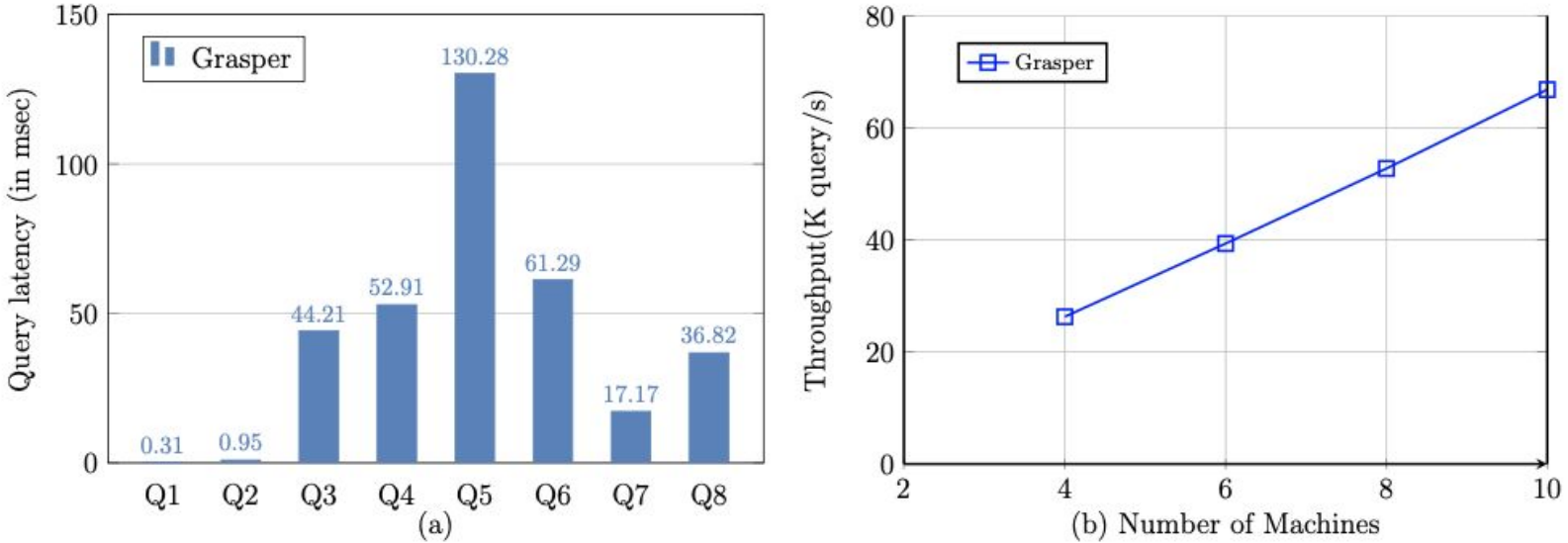


Figure 17: (a) Query latency (in *msec*) of Q1-Q8 on Twitter by Grasper (using 10 machines); (b) Throughput of Grasper on Twitter for {Q1, Q2, Q6}

Adaptive Parallelism & Thread Bounding

- Without Adaptive Parallelism (w/o APC), a message is processed with 1 thread.
- Thread Binding and Load Balancing, TS&LB.

Table 7: Query latency (in *msec*) of Grasper w/ and w/o APC

LDBC	IC1	IC2	IC3	IC4	IS1	IS2	IS3	IS4
Grasper	271	16.7	388	77.3	0.30	2.19	0.91	0.32
w/o APC	469	24.8	666	131	0.51	3.63	1.43	0.54
AMiner	**Q1**	**Q2**	**Q3**	**Q4**	**Q5**	**Q6**	**Q7**	**Q8**
Grasper	0.17	0.42	17.3	45.2	104	28.8	2.32	4.41
w/o APC	0.20	0.62	23.7	59.6	111	35.4	4.50	6.15

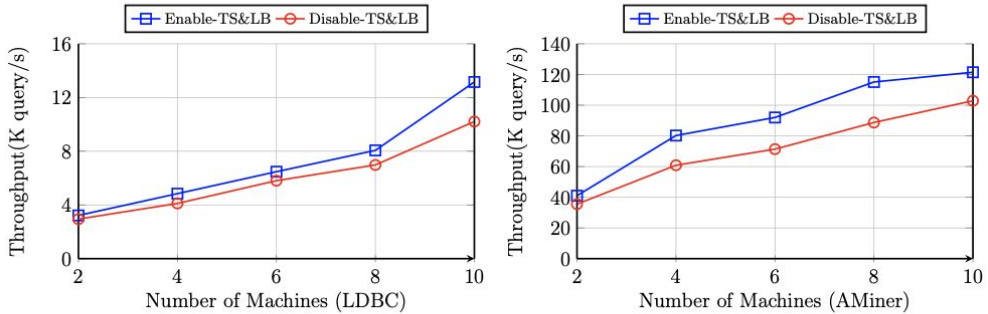


Figure 15: (a) Throughput of Grasper on LDBC for {IS1-IS4}, and (b) throughput of Grasper on AMiner for {Q1, Q2, Q6}, w/ and w/o TS&LB

Comparison With Other Systems

Is this an apple-to-apple comparison?

What can we learn from these graphs?

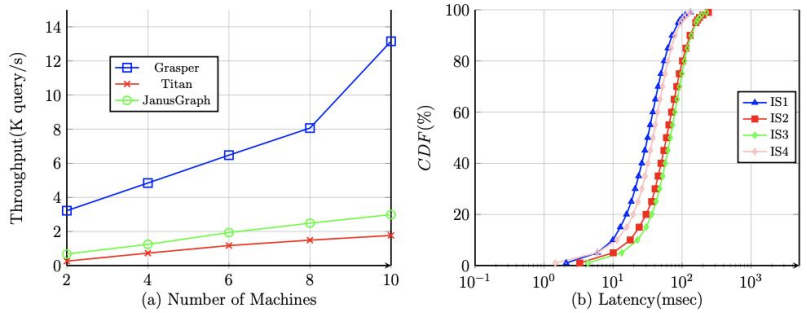


Figure 13: [Best viewed in color] (a) Throughput on LDBC for {IS1-IS4}; (b) CDFs of Grasper's query latency for {IS1-IS4} (using 10 machines)

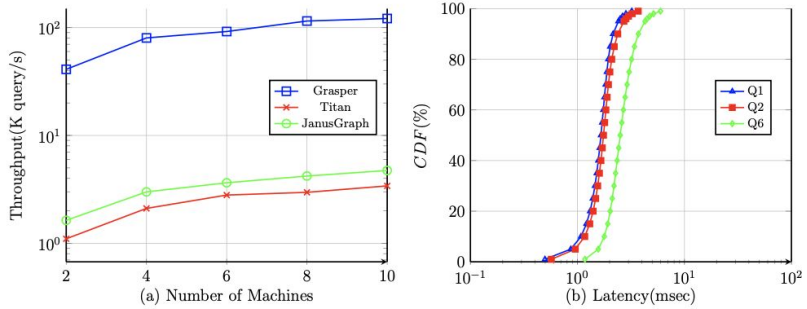


Figure 14: [Best viewed in color] (a) Throughput on AMiner for {Q1, Q2, Q6}; (b) CDFs of Grasper's query latency for {Q1, Q2, Q6} (using 10 machines)

Scalability of Grasper

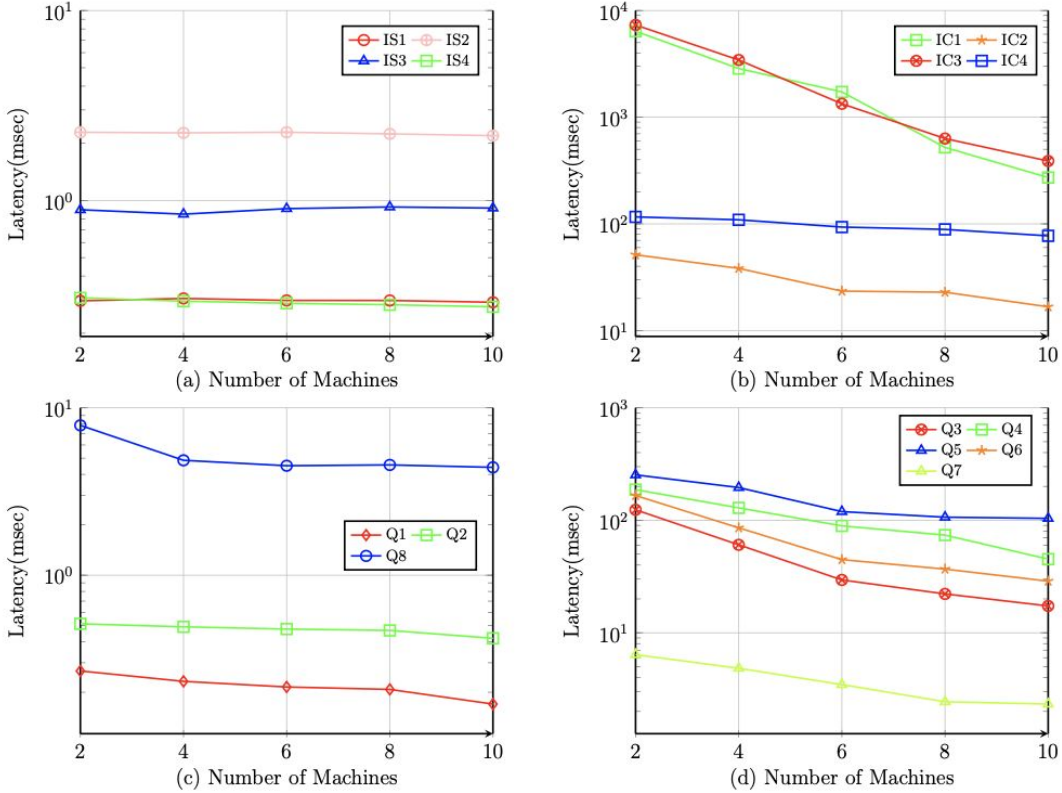


Figure 16: The scalability of Grasper on query latency

Summary

- Grasper:
  - An in-memory system for PGs.
  - Implements the Gremlin query langauge.
  - Data flow for query processing.
  - Expert model for Gremlin operators.
  - RDMA.
  - Work stealing.
  - Query optimization.
- Limitations:
  - Failure handling?
  - How to handle writes? Transactions with ACID semantics?
  - Durable storage? If HDFS is the durable storage then writes must be performed against it.
  - No streaming/dataflow!?!

Figure 10(b) provides more details based on an example query in Figure 8. In Step 2, the *Filter* expert is processing the *hasKey* query step with a parallelism of 2 on query engine 1. It maintains an LRU cache with 1,000 slots and an index for fast retrieval of property keys. Once the execution is completed, the *Filter* expert will send all newly generated messages to the query engines for the processing of next step, where the query engines store the data needed for the next step. Here, the *Filter* expert applies the round-robin strategy to route the messages to idle threads in the *thread region* which the next expert belongs to, for efficient memory access in the same NUMA node. In addition, when we continue to process this query from Step 2 (i.e., *hasKey*) to Step 8 (i.e., *has*), the *Filter* expert will be launched again but with a different parallelism adapted to the workload of Step 8. In Step 4, the *Traversal* expert is activated to